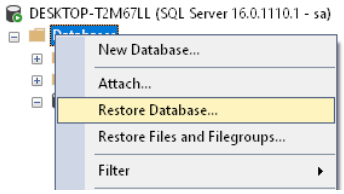
I. BASE DE DATOS 2
1. Restore 2
2. Creación de las Bases de Datos 4
A. STAGE 4
B. DWH 5
3. Creación de objetos 6
A. STAGE 6
B. DWH 12
C. Modelo de la base de datos 23
II. CREACIÓN DE ETL 24
1. Integration Services (SSIS) 24
A. Lógica del ETL (Componentes genéricos) 26
B. Configuración de Conexiones - Global 27
C. Configuración de componentes 29
D. Consideraciones 34
E. Paquetes 34
F. Implementación 55

I. BASE DE DATOS

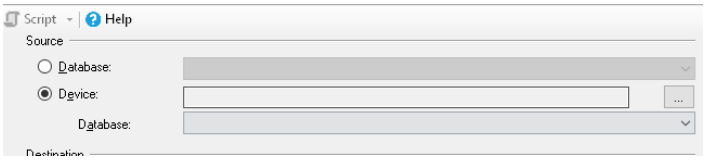
1. Restore

La restauración de una base de datos en SQL Server es un proceso esencial en la gestión de bases de datos, que permite recuperar datos críticos en caso de pérdida o daño de la base de datos original. Este procedimiento implica recuperar una copia de seguridad previamente realizada y volver a colocarla en el servidor de bases de datos, restableciendo así la integridad y la accesibilidad de los datos.

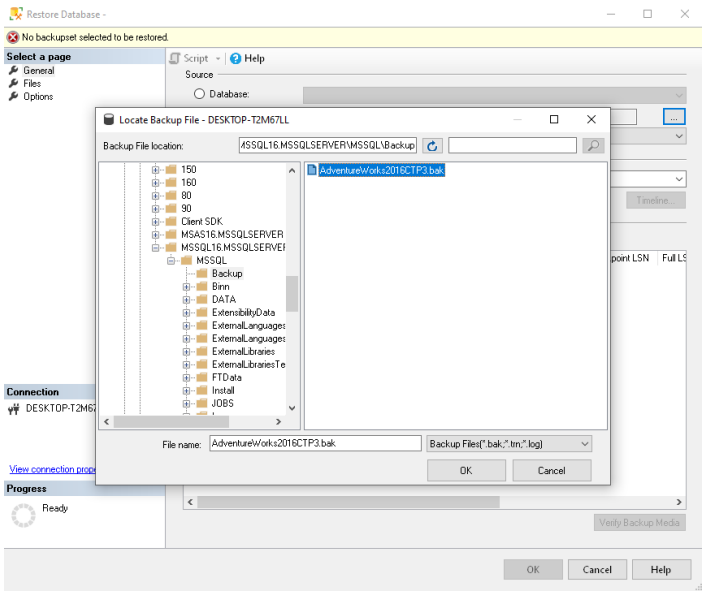
- En el Explorador de objetos, sobre la carpeta "Bases de datos" y selecciona la opción de restauración de base de datos.



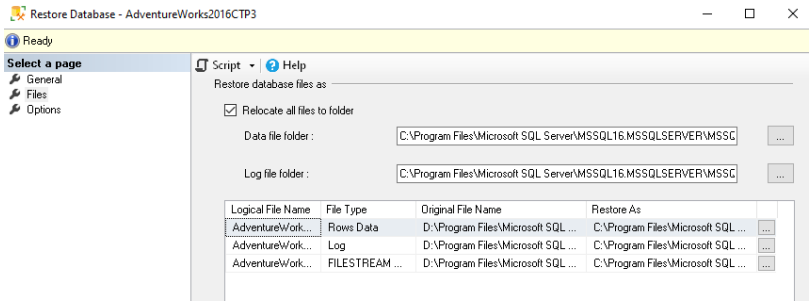
- En la página "Origen", selecciona "Dispositivo" y haz clic en el botón "..." para seleccionar el archivo de copia de seguridad desde donde se restaurará la base de datos.



- En la ventana emergente "Seleccionar dispositivos de copia de seguridad", haz clic en "Agregar" y navega hasta el archivo de copia de seguridad (.bak) que deseas restaurar. Una vez seleccionado, haz clic en "Aceptar".



- En la página "Archivos", puedes configurar diversas opciones, como la ubicación de los archivos de datos y registros. No olvide cambiar las ubicaciones de los archivos según sea la configuración de su entorno.



- Después de restaurar la base de datos, asegúrate de verificar que la restauración se haya realizado correctamente. Puedes hacerlo ejecutando consultas SQL o revisando los datos en el Explorador de objetos de SSMS.

2. Creación de las Bases de Datos

La fase de creación de las bases de datos es un paso crucial en el proceso de construcción de un entorno de almacenamiento de datos eficiente y funcional. En este contexto, se establecen dos bases de datos clave: "STAGE" y "DWH" (almacén de datos).

El código proporcionado establece la creación de la base de datos “STAGE_AW”/ “ DWH_AW”, definiendo sus características de almacenamiento y configuraciones específicas para su funcionamiento adecuado.

A. STAGE

Esta base de datos desempeña un papel fundamental como un intermediario entre las fuentes de datos originales y el destino final, que es el almacén de datos (DWH). En esta etapa, los datos pueden someterse a diversos tratamientos y transformaciones antes de ser cargados en el DWH. Es esencial para garantizar la integridad y la calidad de los datos antes de su consolidación en el almacén principal.

```
CREATE DATABASE [STAGE_AW]
 CONTAINMENT = NONE
 ON  PRIMARY
( NAME = N'STAGE_AW'
,       FILENAME       =       N'C:\Program       Files\Microsoft       SQL
Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\STAGE_AW.mdf'
, SIZE = 102400KB
, MAXSIZE = 302400KB
, FILEGROWTH = 10%
)
 LOG ON
( NAME = N'STAGE_AW_log'
,       FILENAME       =       N'C:\Program       Files\Microsoft       SQL
Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\STAGE_AW_log.ldf'
, SIZE = 102400KB
, MAXSIZE = 302400KB
, FILEGROWTH = 10%
)

 WITH LEDGER = OFF
GO
ALTER DATABASE [STAGE_AW] SET COMPATIBILITY_LEVEL = 160
GO
ALTER DATABASE [STAGE_AW] SET PARAMETERIZATION SIMPLE
GO
ALTER DATABASE [STAGE_AW] SET  READ_WRITE
GO
ALTER DATABASE [STAGE_AW] SET RECOVERY FULL
GO
ALTER DATABASE [STAGE_AW] SET  MULTI_USER
GO
ALTER DATABASE [STAGE_AW] SET PAGE_VERIFY CHECKSUM
GO
USE [STAGE_AW]
GO
IF NOT EXISTS (SELECT name FROM sys.filegroups WHERE is_default=1 AND name
= N'PRIMARY') ALTER DATABASE [STAGE_AW] MODIFY FILEGROUP [PRIMARY] DEFAULT
       GO
```

B. DWH

El almacén de datos (Data Warehouse) es una base de datos centralizada y unificada que almacena datos históricos y actuales de la organización. Aquí es donde se consolidan y organizan los datos provenientes de diversas fuentes, como sistemas transaccionales, aplicaciones empresariales y otros repositorios de datos. El DWH proporciona un entorno optimizado para consultas y análisis, lo que permite a los usuarios de negocio extraer información significativa para la toma de decisiones.

```
CREATE DATABASE [DWH_AW]
 CONTAINMENT = NONE
 ON  PRIMARY
( NAME = N'DWH_AW'
,       FILENAME       =       N'C:\Program       Files\Microsoft       SQL
Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\DWH_AW.mdf'
, SIZE = 8192KB
, MAXSIZE = 102400KB
, FILEGROWTH = 10%
)
 LOG ON
( NAME = N'DWH_AW_log'
,       FILENAME       =       N'C:\Program       Files\Microsoft       SQL
Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\DWH_AW_log.ldf'
, SIZE = 8192KB
, MAXSIZE = 102400KB
, FILEGROWTH = 10%
)

 WITH LEDGER = OFF
GO
ALTER DATABASE [DWH_AW] SET COMPATIBILITY_LEVEL = 160
GO
ALTER DATABASE [DWH_AW] SET PARAMETERIZATION SIMPLE
GO
ALTER DATABASE [DWH_AW] SET  READ_WRITE
GO
ALTER DATABASE [DWH_AW] SET RECOVERY FULL
GO
ALTER DATABASE [DWH_AW] SET  MULTI_USER
GO
ALTER DATABASE [DWH_AW] SET PAGE_VERIFY CHECKSUM
GO
USE [DWH_AW]
GO
IF NOT EXISTS (SELECT name FROM sys.filegroups WHERE is_default=1 AND name
= N'PRIMARY') ALTER DATABASE [DWH_AW] MODIFY FILEGROUP [PRIMARY] DEFAULT
       GO
```

3. Creación de objetos

A. STAGE

Cada una de las tablas del stage representa una etapa clave en el flujo de datos, capturando información crítica relacionada con diferentes aspectos del negocio, como clientes, productos, órdenes de venta y más. Estas tablas están diseñadas para reflejar fielmente la estructura de los datos provenientes de diversas fuentes, lo que facilita su procesamiento y preparación para su posterior análisis.

A continuación, se presenta un resumen de las tablas del stage, cada una con su propia estructura diseñada para capturar y transformar datos específicos necesarios para el análisis empresarial. Estas tablas son la base sobre la cual se construye el conocimiento y la toma de decisiones en el ámbito de Business Intelligence.

```
CREATE SCHEMA [SALES]
GO
```

```
CREATE TABLE [SALES].[StageCurrencyRate](
	[CurrencyRateID] [int] NULL,
	[CurrencyRateDate] [datetime] NOT NULL,
	[FromCurrencyCode] [nchar](3) NOT NULL,
	[ToCurrencyCode] [nchar](3) NOT NULL,
	[AverageRate] [money] NOT NULL,
	[EndOfDayRate] [money] NOT NULL,
	[ModifiedDate] [datetime] NOT NULL
) ON [PRIMARY]
GO
```

```
CREATE TABLE [SALES].[StageCustomer](
	[CustomerID] [int] NOT NULL,
	[BusinessEntityID] [int] NOT NULL,
	[AccountNumber] [varchar](10) NOT NULL,
	[AddressID] [int] NULL,
	[Title] [nvarchar](8) NULL,
	[FirstName] [nvarchar](50) NOT NULL,
	[MiddleName] [nvarchar](50) NULL,
	[LastName] [nvarchar](50) NOT NULL,
	[EmailAddress] [nvarchar](50) NULL,
	[PhoneNumber] [nvarchar](25) NULL,
	[AddressLine1] [nvarchar](60) NOT NULL,
	[AddressLine2] [nvarchar](60) NULL,
	[City] [nvarchar](30) NOT NULL,
	[PostalCode] [nvarchar](15) NOT NULL,
	[StateProvinceCode] [nchar](3) NOT NULL,
	[StateProvinceName] [nvarchar](50) NOT NULL,
	[CountryRegionCode] [nvarchar](3) NOT NULL,
	[CountryRegionName] [nvarchar](50) NOT NULL
) ON [PRIMARY]
GO
```

```
CREATE TABLE [SALES].[StageEmployee](
	[EmployeeID] [int] NOT NULL,
	[NationalIDNumber] [nvarchar](15) NOT NULL,
	[FirstName] [nvarchar](50) NOT NULL,
	[MiddleName] [nvarchar](50) NULL,
	[LastName] [nvarchar](50) NOT NULL,
	[LoginID] [nvarchar](256) NOT NULL,
	[JobTitle] [nvarchar](50) NOT NULL,
	[BirthDate] [date] NOT NULL,
	[MaritalStatus] [nchar](1) NOT NULL,
```

```
	[Gender] [nchar](1) NOT NULL,
	[HireDate] [date] NOT NULL,
	[SalariedFlag] [bit] NOT NULL,
	[CurrentFlag] [bit] NOT NULL,
	[EmailAddress] [nvarchar](50) NULL,
	[PhoneNumber] [nvarchar](25) NOT NULL
) ON [PRIMARY]
GO
```

```
CREATE TABLE [SALES].[StageProduct](
	[ProductID] [int] NULL,
	[Name] [nvarchar](50) NOT NULL,
	[ProductNumber] [nvarchar](25) NOT NULL,
	[MakeFlag] [bit] NOT NULL,
	[FinishedGoodsFlag] [bit] NOT NULL,
	[Color] [nvarchar](15) NULL,
	[SafetyStockLevel] [smallint] NOT NULL,
	[ReorderPoint] [smallint] NOT NULL,
	[StandardCost] [money] NOT NULL,
	[ListPrice] [money] NOT NULL,
	[Size] [nvarchar](5) NULL,
	[SizeUnitMeasureCode] [nchar](3) NULL,
	[WeightUnitMeasureCode] [nchar](3) NULL,
	[Weight] [decimal](8, 2) NULL,
	[DaysToManufacture] [int] NOT NULL,
	[ProductLine] [nchar](2) NULL,
	[Class] [nchar](2) NULL,
	[Style] [nchar](2) NULL,
	[ProductSubcategoryID] [int] NULL,
	[ProductModelID] [int] NULL,
	[SellStartDate] [datetime] NOT NULL,
	[SellEndDate] [datetime] NULL,
	[DiscontinuedDate] [datetime] NULL,
	[ModifiedDate] [datetime] NOT NULL
) ON [PRIMARY]
GO
```

```
CREATE TABLE [SALES].[StageProductCategory](
	[ProductCategoryID] [int] NULL,
	[Name] [nvarchar](50) NOT NULL,
	[rowguid] [uniqueidentifier] NOT NULL,
	[ModifiedDate] [datetime] NOT NULL
) ON [PRIMARY]
GO
```

```
CREATE TABLE [SALES].[StageProductSubcategory](
	[ProductSubcategoryID] [int] NULL,
	[ProductCategoryID] [int] NOT NULL,
	[Name] [nvarchar](50) NOT NULL,
	[rowguid] [uniqueidentifier] NOT NULL,
	[ModifiedDate] [datetime] NOT NULL
) ON [PRIMARY]
GO
```

```
CREATE TABLE [SALES].[StageSalesOrderDetail](
	[SalesOrderID] [int] NOT NULL,
	[SalesOrderDetailID] [int] NULL,
	[ProductID] [int] NOT NULL,
	[SpecialOfferID] [int] NOT NULL,
	[OrderQty] [smallint] NOT NULL,
	[UnitPrice] [money] NOT NULL,
	[UnitPriceDiscount] [money] NOT NULL
) ON [PRIMARY]
GO
```

```
CREATE TABLE [SALES].[StageSalesOrderHeader](
	[SalesOrderID] [int] NULL,
	[RevisionNumber] [tinyint] NOT NULL,
```

```
        [SalesOrderNumber] [nvarchar](25) NOT NULL,
        [PurchaseOrderNumber] [nvarchar](25) NULL,
        [AccountNumber] [nvarchar](15) NULL,
        [OrderDate] [datetime] NOT NULL,
        [DueDate] [datetime] NOT NULL,
        [ShipDate] [datetime] NULL,
        [CustomerID] [int] NOT NULL,
        [EmployeeID] [int] NULL,
        [CurrencyRateID] [int] NULL,
        [Status] [tinyint] NOT NULL,
        [OnlineOrderFlag] [bit] NOT NULL,
        [SubTotal] [money] NOT NULL,
        [TaxAmt] [money] NOT NULL,
        [Freight] [money] NOT NULL
) ON [PRIMARY]
GO
```

```
CREATE TABLE [SALES].[StageSpecialOffer](
        [SpecialOfferID] [int] NULL,
        [Description] [nvarchar](255) NOT NULL,
        [DiscountPct] [smallmoney] NOT NULL,
        [Type] [nvarchar](50) NOT NULL,
        [Category] [nvarchar](50) NOT NULL,
        [StartDate] [datetime] NOT NULL,
        [EndDate] [datetime] NOT NULL,
        [MinQty] [int] NOT NULL,
        [MaxQty] [int] NULL,
        [rowguid] [uniqueidentifier] NOT NULL,
        [ModifiedDate] [datetime] NOT NULL
) ON [PRIMARY]
GO
```

los procedimientos almacenados son especialmente importantes para el procesamiento de datos ETL (Extracción, Transformación y Carga), la generación de informes y el análisis de datos. Permiten ejecutar operaciones repetitivas de manera consistente y confiable, lo que contribuye a la consistencia y la integridad de los datos en todo el sistema.

Los procedimientos almacenados pueden incluir lógica empresarial personalizada, operaciones de limpieza y transformación de datos, así como consultas complejas para la extracción de información significativa. Además, ofrecen ventajas de seguridad al limitar el acceso directo a las tablas de la base de datos y al permitir la ejecución controlada de operaciones específicas a través de interfaces seguras.

```
create procedure [SALES].[sp_StageCurrencyRate]
AS
BEGIN
INSERT INTO [SALES].[StageCurrencyRate]
           ([CurrencyRateID]
           ,[CurrencyRateDate]
           ,[FromCurrencyCode]
           ,[ToCurrencyCode]
           ,[AverageRate]
           ,[EndOfDayRate]
           ,[ModifiedDate])


SELECT
              CurrencyRateID,
              CurrencyRateDate,
```

```
                FromCurrencyCode,
                ToCurrencyCode,
                AverageRate,
                EndOfDayRate,
                ModifiedDate
FROM [AdventureWorks2016CTP3].Sales.CurrencyRate
END

GO
```

```
CREATE PROCEDURE [SALES].[sp_StageCustomer]
AS
BEGIN
INSERT INTO [SALES].[StageCustomer]
           ([CustomerID]
           ,[BusinessEntityID]
           ,[AccountNumber]
           ,[AddressID]
           ,[Title]
           ,[FirstName]
           ,[MiddleName]
           ,[LastName]
           ,[EmailAddress]
           ,[PhoneNumber]
           ,[AddressLine1]
           ,[AddressLine2]
           ,[City]
           ,[PostalCode]
           ,[StateProvinceCode]
           ,[StateProvinceName]
           ,[CountryRegionCode]
           ,[CountryRegionName])

SELECT c.CustomerID,
          p.BusinessEntityID,
          c.AccountNumber,
          bea.AddressID,
          p.Title,
          p.FirstName,
          p.MiddleName ,
          p.LastName ,
          ea.EmailAddress ,
          pp.PhoneNumber ,
          a.AddressLine1 ,
          a.AddressLine2 ,
          a.City ,
          a.PostalCode ,
          s.StateProvinceCode ,
          s.Name AS StateProvinceName ,
          s.CountryRegionCode ,
          cr.Name AS CountryRegionName
FROM [AdventureWorks2016CTP3].Person.Person p
inner join [AdventureWorks2016CTP3].sales.customer c ON (c.PersonID =
p.BusinessEntityID)
left outer join [AdventureWorks2016CTP3].Person.BusinessEntityAddress bea
ON (bea.BusinessEntityID=p.BusinessEntityID AND bea.AddressTypeID = 2) --
billing address
inner join [AdventureWorks2016CTP3].Person.[Address] a ON
(a.AddressID=bea.AddressID)
inner join [AdventureWorks2016CTP3].Person.StateProvince s on
(s.StateProvinceID=a.StateProvinceID)
inner join [AdventureWorks2016CTP3].Person.CountryRegion cr on
(cr.CountryRegionCode = s.CountryRegionCode)
inner join [AdventureWorks2016CTP3].Person.EmailAddress ea ON
(ea.BusinessEntityID = p.BusinessEntityID)
```

```
LEFT OUTER JOIN [AdventureWorks2016CTP3].Person.PersonPhone pp ON
(pp.BusinessEntityID = p.BusinessEntityID)

END
GO
```

```
create procedure [SALES].[sp_StageEmployee]
as
begin
INSERT INTO SALES.StageEmployee
           ([EmployeeID]
           ,[NationalIDNumber]
           ,[FirstName]
           ,[MiddleName]
           ,[LastName]
           ,[LoginID]
           ,[JobTitle]
           ,[BirthDate]
           ,[MaritalStatus]
           ,[Gender]
           ,[HireDate]
           ,[SalariedFlag]
           ,[CurrentFlag]
           ,[EmailAddress]
           ,[PhoneNumber])
   SELECT
       e.BusinessEntityID AS EmployeeID,
       e.NationalIDNumber,
       p.FirstName,
       p.MiddleName,
       p.LastName,
       e.LoginID,
       e.JobTitle,
       e.BirthDate,
       e.MaritalStatus,
       e.Gender,
       e.HireDate,
       e.SalariedFlag,
       e.CurrentFlag,
       em.EmailAddress,
       pp.PhoneNumber
FROM [AdventureWorks2016CTP3].HumanResources.Employee e
INNER      JOIN      [AdventureWorks2016CTP3].Person.Person      p      ON
(p.BusinessEntityID=e.BusinessEntityID)
INNER    JOIN    [AdventureWorks2016CTP3].Person.EmailAddress    em    ON
(em.BusinessEntityID=e.BusinessEntityID)
INNER    JOIN    [AdventureWorks2016CTP3].Person.PersonPhone    pp    ON
(pp.BusinessEntityID=e.BusinessEntityID)

end

GO
```

```
CREATE PROCEDURE [SALES].[sp_StageProductSubcategory]
AS
BEGIN
INSERT INTO SALES.StageProductSubcategory
           ([ProductSubcategoryID]
           ,[ProductCategoryID]
           ,[Name]
           ,[rowguid]
           ,[ModifiedDate])
  SELECT [ProductSubcategoryID]
      ,[ProductCategoryID]
      ,[Name]
      ,[rowguid]
      ,[ModifiedDate]
```

```
  FROM [AdventureWorks2016CTP3].[Production].[ProductSubcategory]
END
GO
```

```
CREATE PROCEDURE [SALES].[sp_StageSalesOrderDetail]
AS
BEGIN
INSERT INTO SALES.StageSalesOrderDetail
           ([SalesOrderID]
           ,[SalesOrderDetailID]
           ,[ProductID]
           ,[SpecialOfferID]
           ,[OrderQty]
           ,[UnitPrice]
           ,[UnitPriceDiscount])
SELECT
       SalesOrderID,
       SalesOrderDetailID,
       ProductID,
       SpecialOfferID,
       OrderQty,
       UnitPrice ,
       UnitPriceDiscount
FROM [AdventureWorks2016CTP3].Sales.SalesOrderDetail
end
GO
```

```
CREATE PROCEDURE [SALES].[sp_StageSalesOrderHeader]
AS
BEGIN
INSERT INTO SALES.StageSalesOrderHeader
           ([SalesOrderID]
           ,[RevisionNumber]
           ,[SalesOrderNumber]
           ,[PurchaseOrderNumber]
           ,[AccountNumber]
           ,[OrderDate]
           ,[DueDate]
           ,[ShipDate]
           ,[CustomerID]
           ,[EmployeeID]
           ,[CurrencyRateID]
           ,[Status]
           ,[OnlineOrderFlag]
           ,[SubTotal]
           ,[TaxAmt]
           ,[Freight])

SELECT
       SalesOrderID,
       RevisionNumber,
       SalesOrderNumber,
       PurchaseOrderNumber ,
       AccountNumber,
       OrderDate,
       DueDate,
       ShipDate,
       CustomerID,
       isnull(SalesPersonID,1) AS EmployeeID,
       isnull(CurrencyRateID,1) as CurrencyRateID,
       [Status],
       OnlineOrderFlag,
       SubTotal,
       TaxAmt ,
       Freight
FROM [AdventureWorks2016CTP3].Sales.SalesOrderHeader
```

```
end
GO
```

```
CREATE PROCEDURE [SALES].[sp_StageSpecialOffer]
AS
BEGIN
INSERT INTO SALES.StageSpecialOffer
           ([SpecialOfferID]
           ,[Description]
           ,[DiscountPct]
           ,[Type]
           ,[Category]
           ,[StartDate]
           ,[EndDate]
           ,[MinQty]
           ,[MaxQty]
           ,[rowguid]
           ,[ModifiedDate])

SELECT
              SpecialOfferID,
              Description,
              DiscountPct,
              Type,
              Category,
              StartDate ,
              EndDate ,
              MinQty,
              MaxQty,
              rowguid,
              ModifiedDate
FROM [AdventureWorks2016CTP3].Sales.SpecialOffer

end
GO
```

B. DWH

La tabla DIM_AUDIT desempeña un papel crucial en el entorno de Business Intelligence (BI) y en los procesos de ETL (Extract, Transform, Load). Aquí hay algunas razones que destacan la importancia de esta tabla:

- **Auditoría de Procesos:** La tabla DIM_AUDIT registra información detallada sobre cada ejecución de los paquetes de ETL. Esto incluye la fecha y hora de inicio y finalización de cada ejecución, así como el éxito o fracaso de esta. Esta auditoría proporciona una trazabilidad completa de los procesos de carga de datos, lo que es esencial para la gestión de la calidad y la resolución de problemas.
- **Seguimiento de Cambios:** Al incluir campos como TableName y los recuentos de filas inicial y final, DIM_AUDIT permite rastrear los cambios en las tablas de destino durante cada ejecución del paquete ETL. Esto facilita la identificación de discrepancias y el análisis de tendencias en el volumen de datos a lo largo del tiempo.
- **Diagnóstico de Errores:** Cuando se produce un error durante el proceso de carga de datos, la tabla DIM_AUDIT registra información valiosa, como el nombre de la tabla afectada, el momento en que ocurrió el error y si la ejecución fue exitosa o no. Estos

detalles son fundamentales para diagnosticar y solucionar problemas de manera oportuna, minimizando el tiempo de inactividad y asegurando la integridad de los datos.

- **Mejora de Desempeño:** El análisis de los datos almacenados en DIM_AUDIT puede proporcionar información útil sobre el rendimiento de los paquetes de ETL y los procesos de carga de datos. Identificar cuellos de botella y áreas de mejora ayuda a optimizar la eficiencia operativa y a cumplir con los objetivos de tiempo de ejecución.
- **Cumplimiento Normativo:** En entornos regulados, como el sector financiero o de atención médica, mantener un registro detallado de todas las actividades relacionadas con los datos es esencial para cumplir con los requisitos normativos y las auditorías externas.

```
CREATE TABLE [dbo].[DIM_AUDIT](
	[AuditKey] [int] IDENTITY(1,1) NOT NULL,
	[TableName] [nvarchar](100) NULL,
	[PkgName] [nvarchar](50) NOT NULL,
	[PkgGUID] [uniqueidentifier] NULL,
	[ExecStartDT] [datetime] NOT NULL,
	[ExecStopDT] [datetime] NULL,
	[ExecutionInstanceGUID] [uniqueidentifier] NULL,
	[ExtractRowCnt] [bigint] NULL,
	[TableInitialRowCnt] [bigint] NULL,
	[TableFinalRowCnt] [bigint] NULL,
	[SuccessfulProcessingInd] [nchar](1) NOT NULL
) ON [PRIMARY]
GO

ALTER TABLE [dbo].[DIM_AUDIT] ADD  DEFAULT ('Unknown') FOR [TableName]
GO

ALTER TABLE [dbo].[DIM_AUDIT] ADD  DEFAULT ('Unknown') FOR [PkgName]
GO

ALTER TABLE [dbo].[DIM_AUDIT] ADD  DEFAULT (getdate()) FOR [ExecStartDT]
GO

ALTER TABLE [dbo].[DIM_AUDIT] ADD  DEFAULT ('N') FOR [SuccessfulProcessingInd]
GO
```

A continuación, se detalla el conjunto de tablas correspondientes a la lógica dentro del DWH_AW.

```
CREATE SCHEMA [Sales]
GO

CREATE TABLE [Sales].[ErrorFactSales](
	[SalesOrderID] [int] NULL,
	[SalesOrderDetailID] [int] NULL,
	[ProductID] [int] NULL,
	[SpecialOfferID] [int] NULL,
	[OrderQty] [smallint] NULL,
	[UnitPrice] [money] NULL,
	[UnitPriceDiscount] [money] NULL,
	[RevisionNumber] [tinyint] NULL,
	[SalesOrderNumber] [nvarchar](25) NULL,
```

```
	[PurchaseOrderNumber] [nvarchar](25) NULL,
	[AccountNumber] [nvarchar](15) NULL,
	[OrderDate] [datetime] NULL,
	[DueDate] [datetime] NULL,
	[ShipDate] [datetime] NULL,
	[CustomerID] [int] NULL,
	[EmployeeID] [int] NULL,
	[CurrencyRateID] [int] NULL,
	[Status] [tinyint] NULL,
	[OnlineOrderFlag] [bit] NULL,
	[SubTotal] [money] NULL,
	[TaxAmt] [money] NULL,
	[Freight] [money] NULL,
	[ProductKey] [int] NULL,
	[OrderDateKey] [int] NULL,
	[DueDateKey] [int] NULL,
	[ShipDateKey] [int] NULL,
	[CustomerKey] [int] NULL,
	[EmployeeKey] [int] NULL,
	[CurrencyKey] [int] NULL
) ON [PRIMARY]
GO

CREATE SCHEMA [GENERICO]
GO

CREATE TABLE [GENERICO].[DimFecha](
	[FechaKey] [int] NOT NULL,
	[Fecha] [datetime] NOT NULL,
	[Dia_Semana] [tinyint] NOT NULL,
	[Nombre_Dia_Semana] [nvarchar](11) NOT NULL,
	[Dia_Mes] [tinyint] NOT NULL,
	[Dia_Anio] [smallint] NOT NULL,
	[Nombre_Mes] [nvarchar](10) NOT NULL,
	[Mes_Anio] [tinyint] NOT NULL,
	[Semana_Anio] [tinyint] NOT NULL,
	[Anio] [smallint] NOT NULL,
	[Bimestre] [tinyint] NOT NULL,
	[Trimestre] [tinyint] NOT NULL,
	[Cuatrimestre] [tinyint] NOT NULL,
	[Semestre] [tinyint] NOT NULL,
	[Visible] [smallint] NOT NULL,
 CONSTRAINT [PK_DimFecha] PRIMARY KEY CLUSTERED
(
	[FechaKey] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF)
ON [PRIMARY]
) ON [PRIMARY]
GO



CREATE TABLE [Sales].[DimCurrency](
	[CurrencyKey] [int] IDENTITY(1,1) NOT NULL,
	[CurrencyRateID] [int] NULL,
	[CurrencyRateDate] [datetime] NOT NULL,
	[FromCurrencyCode] [nchar](3) NOT NULL,
	[ToCurrencyCode] [nchar](3) NOT NULL,
 CONSTRAINT [PK_dbo.DimCurrency] PRIMARY KEY CLUSTERED
(
	[CurrencyKey] ASC
```

```
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF)
ON [PRIMARY]
) ON [PRIMARY]
GO


CREATE TABLE [Sales].[DimCustomer](
	[CustomerKey] [int] IDENTITY(1,1) NOT NULL,
	[CustomerID] [int] NOT NULL,
	[AccountNumber] [varchar](10) NOT NULL,
	[AddressID] [int] NULL,
	[FirstName] [nvarchar](50) NOT NULL,
	[MiddleName] [nvarchar](50) NULL,
	[LastName] [nvarchar](50) NOT NULL,
	[EmailAddress] [nvarchar](50) NULL,
	[PhoneNumber] [nvarchar](25) NULL,
	[AddressLine1] [nvarchar](60) NOT NULL,
	[AddressLine2] [nvarchar](60) NULL,
	[City] [nvarchar](30) NOT NULL,
	[PostalCode] [nvarchar](15) NOT NULL,
	[StateProvinceCode] [nchar](3) NOT NULL,
	[StateProvinceName] [nvarchar](50) NOT NULL,
	[CountryRegionCode] [nvarchar](3) NOT NULL,
	[CountryRegionName] [nvarchar](50) NULL,
 CONSTRAINT [PK_dbo.DimCustomer] PRIMARY KEY CLUSTERED
(
	[CustomerKey] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF)
ON [PRIMARY]
) ON [PRIMARY]
GO


CREATE TABLE [Sales].[DimEmployee](
	[EmployeeKey] [int] IDENTITY(1,1) NOT NULL,
	[EmployeeID] [int] NOT NULL,
	[NationalIDNumber] [nvarchar](15) NOT NULL,
	[FirstName] [nvarchar](50) NOT NULL,
	[MiddleName] [nvarchar](50) NULL,
	[LastName] [nvarchar](50) NOT NULL,
	[LoginID] [nvarchar](256) NOT NULL,
	[JobTitle] [nvarchar](50) NOT NULL,
	[BirthDate] [date] NOT NULL,
	[MaritalStatus] [nchar](1) NOT NULL,
	[Gender] [nchar](1) NOT NULL,
	[HireDate] [date] NOT NULL,
	[SalariedFlag] [bit] NOT NULL,
	[CurrentFlag] [bit] NOT NULL,
	[EmailAddress] [nvarchar](50) NULL,
	[PhoneNumber] [nvarchar](25) NOT NULL,
 CONSTRAINT [PK_dbo.DimEmployee] PRIMARY KEY CLUSTERED
(
	[EmployeeKey] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF)
ON [PRIMARY]
) ON [PRIMARY]
GO
```

```sql
CREATE TABLE [Sales].[DimProduct](
	[ProductKey] [int] IDENTITY(1,1) NOT NULL,
	[ProductID] [int] NULL,
	[Name] [nvarchar](50) NOT NULL,
	[ProductNumber] [nvarchar](25) NOT NULL,
	[Color] [nvarchar](15) NULL,
	[ListPrice] [money] NOT NULL,
	[Size] [nvarchar](5) NULL,
	[Weight] [decimal](8, 2) NULL,
	[Class] [nchar](2) NULL,
	[Style] [nchar](2) NULL,
	[ProductSubcategoryID] [int] NULL,
	[ProductCategoryID] [int] NULL,
	[Subcategory] [nvarchar](50) NULL,
	[Category] [nvarchar](50) NULL,
 CONSTRAINT [PK_dbo.DimProduct] PRIMARY KEY CLUSTERED
(
	[ProductKey] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF)
ON [PRIMARY]
) ON [PRIMARY]
GO


CREATE TABLE [Sales].[FactSales](
	[SalesOrderID] [int] NOT NULL,
	[SalesOrderDetailID] [int] NULL,
	[CustomerKey] [int] NOT NULL,
	[EmployeeKey] [int] NOT NULL,
	[CurrencyKey] [int] NOT NULL,
	[ProductKey] [int] NOT NULL,
	[OrderDateKey] [int] NOT NULL,
	[DueDateKey] [int] NOT NULL,
	[ShipDateKey] [int] NOT NULL,
	[SpecialOfferID] [int] NOT NULL,
	[OrderQty] [smallint] NOT NULL,
	[UnitPrice] [money] NOT NULL,
	[UnitPriceDiscount] [money] NOT NULL,
	[RevisionNumber] [tinyint] NOT NULL,
	[SalesOrderNumber] [nvarchar](25) NOT NULL,
	[PurchaseOrderNumber] [nvarchar](25) NULL,
	[AccountNumber] [nvarchar](15) NULL,
	[Status] [tinyint] NOT NULL,
	[OnlineOrderFlag] [bit] NOT NULL,
	[SubTotal] [money] NOT NULL,
	[TaxAmt] [money] NOT NULL,
	[Freight] [money] NOT NULL
) ON [PRIMARY]
GO

Create View [Sales].[vw_StageProduct]
as
SELECT p.[ProductID]
      ,p.[Name]
      ,p.[ProductNumber]
      ,p.[Color]
      ,p.[ListPrice]
      ,p.[Size]
      ,p.[Weight]
      ,p.[Class]
      ,p.[Style]
      ,isnull(p.[ProductSubcategoryID],-1) as ProductSubcategoryID
```

```
        ,isnull(sc.[ProductCategoryID],-1) as ProductCategoryID
      ,isnull(sc.[Name],'Sin Datos') as Subcategory
        ,isnull(pc.[Name],'Sin Datos') as Category
  FROM [Demo_Stage_AW].[SALES].[StageProduct] p
  left join [Demo_Stage_AW].[SALES].[StageProductSubcategory] sc on
(p.ProductSubcategoryID = sc.ProductSubcategoryID)
  left join [Demo_Stage_AW].[SALES].[StageProductCategory] pc on
(pc.[ProductCategoryID] = sc.[ProductCategoryID])
GO
```

La integridad referencial es un aspecto fundamental en el diseño de bases de datos relacionales, garantizando que los datos mantengan coherencia y consistencia entre las diferentes tablas. En el contexto de Business Intelligence (BI), donde se manejan grandes volúmenes de datos para análisis y generación de informes, establecer relaciones entre tablas es crucial para obtener información precisa y significativa.

la creación de restricciones de clave externa (foreign key constraints) juega un papel fundamental. Estas restricciones aseguran que los valores en una columna de una tabla (llamada clave externa) coincidan con los valores en otra columna (llamada clave primaria o única) en otra tabla. Esto permite establecer relaciones entre tablas y mantener la integridad referencial de los datos.

```
ALTER TABLE [Sales].[FactSales]  WITH CHECK ADD  CONSTRAINT
[FK_FactSales_DimCurrency] FOREIGN KEY([CurrencyKey])
REFERENCES [Sales].[DimCurrency] ([CurrencyKey])
GO
ALTER TABLE [Sales].[FactSales] CHECK CONSTRAINT [FK_FactSales_DimCurrency]
GO
ALTER TABLE [Sales].[FactSales]  WITH CHECK ADD  CONSTRAINT
[FK_FactSales_DimCustomer] FOREIGN KEY([CustomerKey])
REFERENCES [Sales].[DimCustomer] ([CustomerKey])
GO
ALTER TABLE [Sales].[FactSales] CHECK CONSTRAINT [FK_FactSales_DimCustomer]
GO
ALTER TABLE [Sales].[FactSales]  WITH CHECK ADD  CONSTRAINT
[FK_FactSales_DimEmployee] FOREIGN KEY([EmployeeKey])
REFERENCES [Sales].[DimEmployee] ([EmployeeKey])
GO
ALTER TABLE [Sales].[FactSales] CHECK CONSTRAINT [FK_FactSales_DimEmployee]
GO
ALTER TABLE [Sales].[FactSales]  WITH CHECK ADD  CONSTRAINT
[FK_FactSales_DimFecha] FOREIGN KEY([OrderDateKey])
REFERENCES [GENERICO].[DimFecha] ([FechaKey])
GO
ALTER TABLE [Sales].[FactSales] CHECK CONSTRAINT [FK_FactSales_DimFecha]
GO
ALTER TABLE [Sales].[FactSales]  WITH CHECK ADD  CONSTRAINT
[FK_FactSales_DimFecha1] FOREIGN KEY([DueDateKey])
REFERENCES [GENERICO].[DimFecha] ([FechaKey])
GO
ALTER TABLE [Sales].[FactSales] CHECK CONSTRAINT [FK_FactSales_DimFecha1]
GO
ALTER TABLE [Sales].[FactSales]  WITH CHECK ADD  CONSTRAINT
[FK_FactSales_DimFecha2] FOREIGN KEY([ShipDateKey])
REFERENCES [GENERICO].[DimFecha] ([FechaKey])
```

```
GO
ALTER TABLE [Sales].[FactSales] CHECK CONSTRAINT [FK_FactSales_DimFecha2]
GO
ALTER TABLE [Sales].[FactSales]  WITH CHECK ADD  CONSTRAINT
[FK_FactSales_DimProduct] FOREIGN KEY([ProductKey])
REFERENCES [Sales].[DimProduct] ([ProductKey])
GO
ALTER TABLE [Sales].[FactSales] CHECK CONSTRAINT [FK_FactSales_DimProduct]
GO
```

Finalmente, creamos los procedimientos (logica de negocio).

```
CREATE PROCEDURE [Sales].[sp_DimCurrency]
as
BEGIN
MERGE [Sales].DimCurrency AS TARGET
USING [Demo_Stage_AW].[SALES].[StageCurrencyRate] AS SOURCE
     ON (TARGET.CurrencyRateID = SOURCE.CurrencyRateID )
          WHEN MATCHED AND  TARGET.[FromCurrencyCode] <>
SOURCE.[FromCurrencyCode] OR TARGET.[ToCurrencyCode] <> SOURCE.[ToCurrencyCode]
                            OR TARGET.[CurrencyRateDate] <>
SOURCE.[CurrencyRateDate]  THEN
             UPDATE SET TARGET.[FromCurrencyCode] = SOURCE.[FromCurrencyCode],
                        TARGET.[ToCurrencyCode] = SOURCE.[ToCurrencyCode],
                             TARGET.[CurrencyRateDate] =
SOURCE.[CurrencyRateDate]
          WHEN NOT MATCHED BY TARGET THEN
    INSERT ( [CurrencyRateID]
                    ,[CurrencyRateDate]
                    ,[FromCurrencyCode]
                    ,[ToCurrencyCode])
VALUES (SOURCE.[CurrencyRateID]
      ,SOURCE.[CurrencyRateDate]
      ,SOURCE.[FromCurrencyCode]
      ,SOURCE.[ToCurrencyCode]);
END
GO


CREATE PROCEDURE [Sales].[sp_DimCustomer]
as
begin
INSERT INTO Sales.DimCustomer
           ([CustomerID]
           ,[AccountNumber]
           ,[AddressID]
           ,[FirstName]
           ,[MiddleName]
           ,[LastName]
           ,[EmailAddress]
           ,[PhoneNumber]
           ,[AddressLine1]
           ,[AddressLine2]
           ,[City]
           ,[PostalCode]
           ,[StateProvinceCode]
           ,[StateProvinceName]
           ,[CountryRegionCode]
                ,[CountryRegionName])
  SELECT [CustomerID]
```

```
      ,[AccountNumber]
      ,[AddressID]
      ,[FirstName]
      ,[MiddleName]
      ,[LastName]
      ,[EmailAddress]
      ,[PhoneNumber]
      ,[AddressLine1]
      ,[AddressLine2]
      ,[City]
      ,[PostalCode]
      ,[StateProvinceCode]
      ,[StateProvinceName]
      ,[CountryRegionCode]
      ,[CountryRegionName]
  FROM [Demo_Stage_AW].[SALES].[StageCustomer]
  where [CustomerID] not in (select [CustomerID] from Sales.DimCustomer)



end
GO

create procedure [Sales].[sp_DimEmployee]
as
begin
INSERT INTO [Sales].[DimEmployee]
           ([EmployeeID]
           ,[NationalIDNumber]
           ,[FirstName]
           ,[MiddleName]
           ,[LastName]
           ,[LoginID]
           ,[JobTitle]
           ,[BirthDate]
           ,[MaritalStatus]
           ,[Gender]
           ,[HireDate]
           ,[SalariedFlag]
           ,[CurrentFlag]
           ,[EmailAddress]
           ,[PhoneNumber])
  SELECT [EmployeeID]
      ,[NationalIDNumber]
      ,[FirstName]
      ,[MiddleName]
      ,[LastName]
      ,[LoginID]
      ,[JobTitle]
      ,[BirthDate]
      ,[MaritalStatus]
      ,[Gender]
      ,[HireDate]
      ,[SalariedFlag]
      ,[CurrentFlag]
      ,[EmailAddress]
      ,[PhoneNumber]
  FROM [Demo_Stage_AW].[SALES].[StageEmployee]
  where [EmployeeID] not in (select [EmployeeID] from [Sales].[DimEmployee])

  end
GO
```

```
CREATE procedure [Sales].[sp_DimProduct]
AS
BEGIN



MERGE [Sales].[DimProduct] AS TARGET
USING [Sales].[vw_StageProduct] AS SOURCE
     ON (TARGET.ProductID = SOURCE.ProductID )
          WHEN MATCHED AND  TARGET.[Name] <> SOURCE.[Name] OR
TARGET.[Subcategory] <> SOURCE.[Subcategory] OR TARGET.[Category] <>
SOURCE.[Category] THEN
              UPDATE SET TARGET.[Name] = SOURCE.[Name],
                         TARGET.[Subcategory] = SOURCE.[Subcategory],
                              TARGET.[Category] = SOURCE.[Category]
          WHEN NOT MATCHED BY TARGET THEN
    INSERT ([ProductID]
           ,[Name]
           ,[ProductNumber]
           ,[Color]
           ,[ListPrice]
           ,[Size]
           ,[Weight]
           ,[Class]
           ,[Style]
           ,[ProductSubcategoryID]
           ,[ProductCategoryID]
           ,[Subcategory]
           ,[Category])


VALUES (SOURCE.[ProductID]
      ,SOURCE.[Name]
      ,SOURCE.[ProductNumber]
      ,SOURCE.[Color]
      ,SOURCE.[ListPrice]
      ,SOURCE.[Size]
      ,SOURCE.[Weight]
      ,SOURCE.[Class]
      ,SOURCE.[Style]
      ,SOURCE.ProductSubcategoryID
         ,SOURCE.ProductCategoryID
      ,SOURCE.Subcategory
         ,SOURCE.Category);

  END
GO

CREATE procedure [Sales].[sp_FactSales]
as
BEGIN

INSERT INTO [Sales].[FactSales]
           ([SalesOrderID]
           ,[SalesOrderDetailID]
           ,[CustomerKey]
           ,[EmployeeKey]
           ,[CurrencyKey]
           ,[ProductKey]
           ,[OrderDateKey]
           ,[DueDateKey]
```

```
           ,[ShipDateKey]
           ,[SpecialOfferID]
           ,[OrderQty]
           ,[UnitPrice]
           ,[UnitPriceDiscount]
           ,[RevisionNumber]
           ,[SalesOrderNumber]
           ,[PurchaseOrderNumber]
           ,[AccountNumber]
           ,[Status]
           ,[OnlineOrderFlag]
           ,[SubTotal]
           ,[TaxAmt]
           ,[Freight])

SELECT d.[SalesOrderID]
      ,d.[SalesOrderDetailID]
         ----
         --,o.[CustomerID]
         ,isnull(c.CustomerKey,-1) as CustomerKey

      --,o.[EmployeeID]
         ,e.EmployeeKey

      --,o.[CurrencyRateID]
         ,cr.CurrencyKey

      --,d.[ProductID]
         ,p.ProductKey

         --,o.[OrderDate]
         ,d1.FechaKey
      --,o.[DueDate]
         ,d2.FechaKey
      --,o.[ShipDate]
         ,d3.FechaKey
         ------
      ,d.[SpecialOfferID]
      ,d.[OrderQty]
      ,d.[UnitPrice]
      ,d.[UnitPriceDiscount]

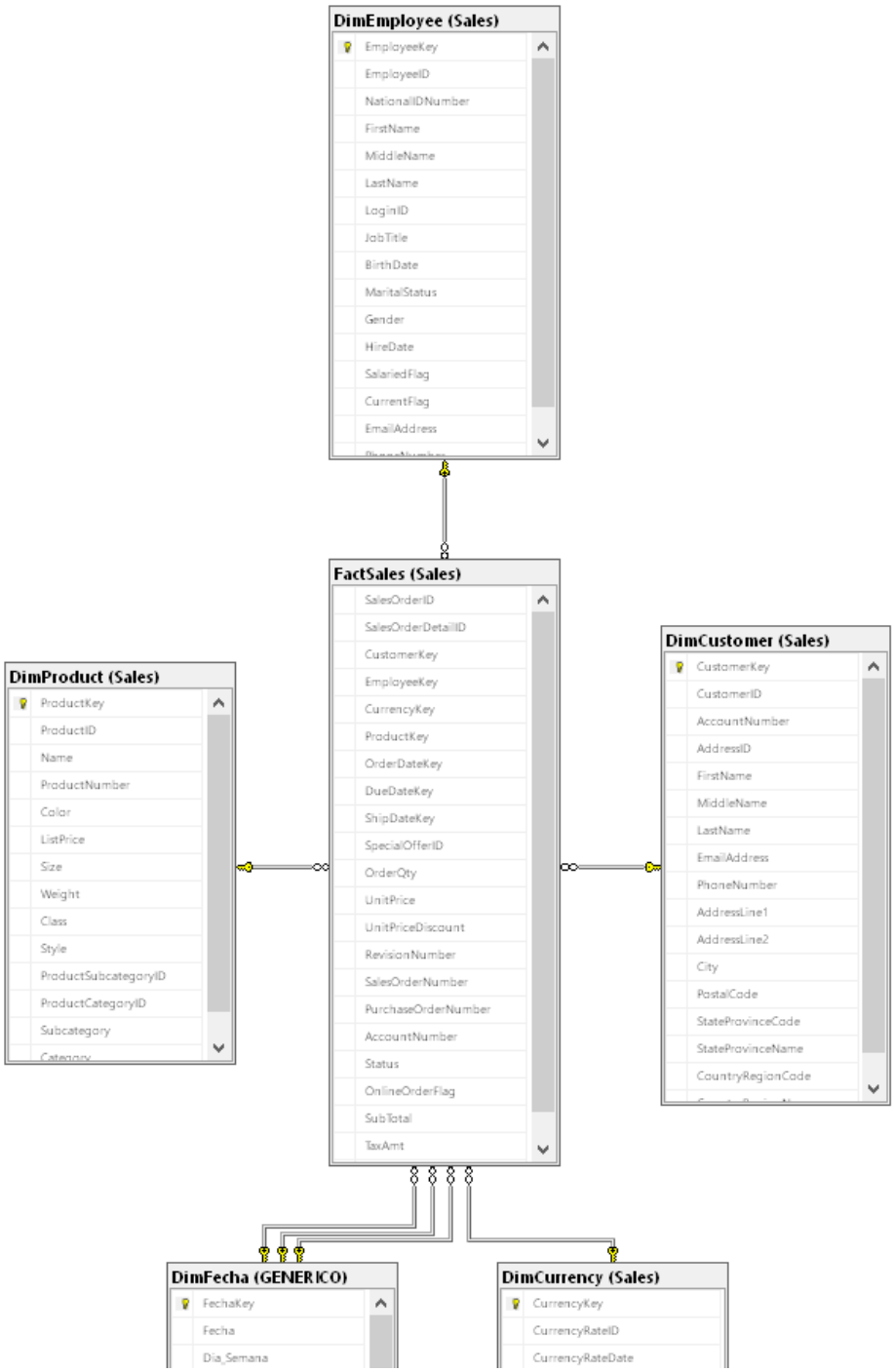
         ,o.[RevisionNumber]
      ,o.[SalesOrderNumber]
      ,o.[PurchaseOrderNumber]
      ,o.[AccountNumber]


      ,o.[Status]
      ,o.[OnlineOrderFlag]
      ,o.[SubTotal]
      ,o.[TaxAmt]
      ,o.[Freight]

  FROM [Demo_Stage_AW].[SALES].[StageSalesOrderDetail] d
  inner join [Demo_Stage_AW].[SALES].[StageSalesOrderHeader] o
  on d.SalesOrderID = o.SalesOrderID
  left join [Sales].[DimCustomer] c
  on c.CustomerID = o.[CustomerID]
  inner join [Sales].[DimEmployee] e
  on e.EmployeeID =o.EmployeeID
  inner join  [Sales].[DimCurrency] cr
```

```
  on cr.CurrencyRateID = o.CurrencyRateID
  inner join [Sales].[DimProduct] p
  on p.ProductID = d.ProductID
  inner join [GENERICO].[DimFecha] d1
  on d1.FechaKey = CAST((CONVERT(VARCHAR(8),o.[OrderDate],112)) AS INT)
  inner join [GENERICO].[DimFecha] d2
  on d2.FechaKey = CAST((CONVERT(VARCHAR(8),o.[DueDate],112)) AS INT)
  inner join [GENERICO].[DimFecha] d3
  on d3.FechaKey = CAST((CONVERT(VARCHAR(8),o.[ShipDate],112)) AS INT)
END
GO
```

C. Modelo de la base de datos



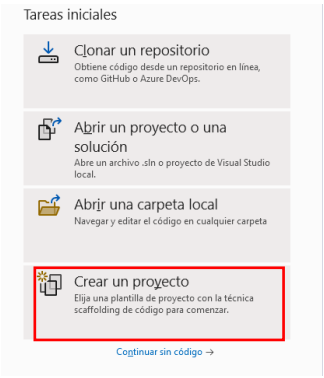
II. CREACIÓN DE ETL

La creación de procesos de ETL (Extract, Transform, Load) es un componente vital en el desarrollo de un sistema de Business Intelligence (BI) eficaz y dinámico. Entre las herramientas más utilizadas para este propósito se encuentra Integration Services (SSIS) de Microsoft, que proporciona un entorno robusto para diseñar, implementar y administrar flujos de trabajo de ETL. Si bien herramientas como Integration Services (SSIS) ofrecen soluciones robustas para ETL, el uso de Python como alternativa proporciona una flexibilidad adicional y una capacidad de personalización que pueden ser beneficiosas en ciertos casos.

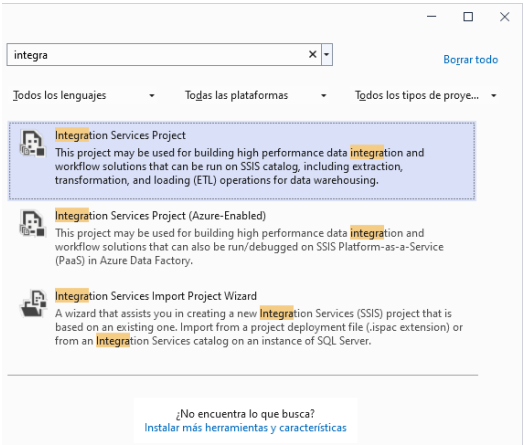
1. Integration Services (SSIS)

En esta sección, exploraremos el uso de Integration Services (SSIS) como herramienta principal para la creación de procesos de ETL en el contexto del proyecto de BI. SSIS ofrece una variedad de componentes y funcionalidades que permiten la extracción de datos desde diversas fuentes, la transformación según las necesidades del negocio, y la carga eficiente de los datos en el almacén de datos. Crear una solución de ETL (Extract, Transform, Load) con Integration Services (SSIS) implica una serie de pasos clave que cubren desde el diseño hasta la implementación y monitoreo de los paquetes de flujo de datos. A continuación, una guía paso a paso para crear una solución de ETL con SSIS:

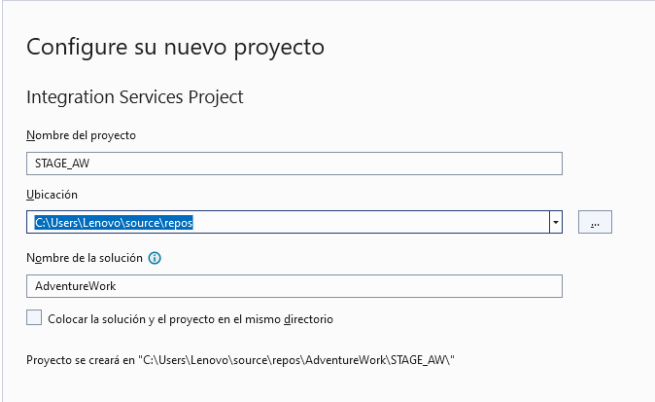
- Accedemos a visual studio (se recomienda las versiones 2019 – 2022), creamos un nuevo proyecto.



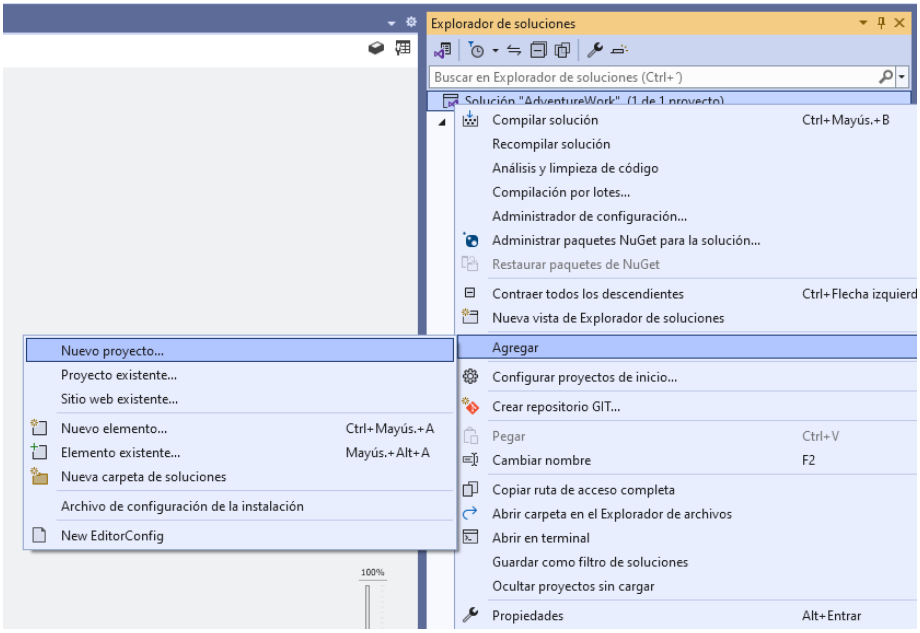
- Usamos el buscador para acceder al componente integration servies project



- El componente solicitará tres elementos esenciales:
    - Nombre del proyecto: Especifica un nombre descriptivo para el proyecto, por ejemplo, "STAGE_AW".
    - Ruta de guardado: Indica la ubicación donde se almacenarán los archivos del proyecto. Se recomienda seleccionar una carpeta especializada para organizar el trabajo de manera eficiente.
    - Nombre de la solución: Se distingue del nombre del proyecto y representa la agrupación de uno o varios proyectos relacionados. Por ejemplo, "AdventureWork" podría ser el nombre de la solución que incluye varios proyectos relacionados con la empresa "Adventure Works"

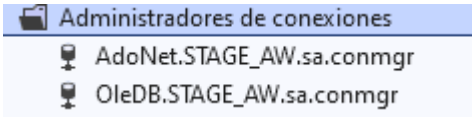


- Después de haber establecido la solución y el primer proyecto de manera exitosa, el siguiente paso implica agregar un nuevo proyecto a la solución. Este nuevo proyecto se denominará "DWH_AW", reflejando su relación con el almacén de datos del proyecto "AdventureWork".
- Los pasos para seguir son similares al inicial, solo se debe establecer el tipo de componente (integration services) y el nombre del proyecto.

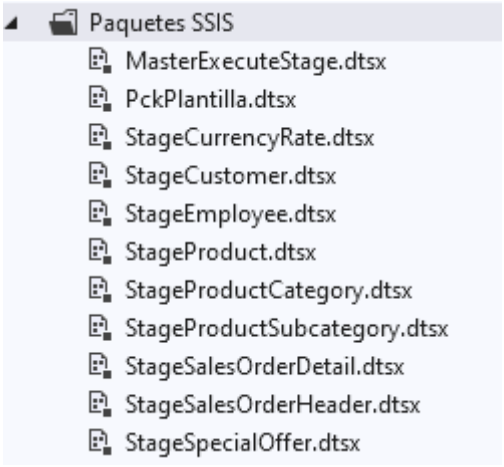


A. Lógica del ETL (Componentes genéricos)

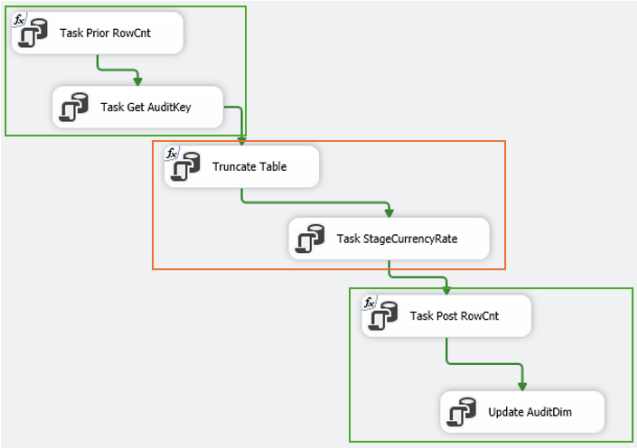
- Permite establecer conexiones a diversas fuentes de datos como bases de datos SQL Server, archivos planos, fuentes OLE DB, conexiones ODBC, entre otras.



- Contienen múltiples tareas de flujo de control, como tareas de extracción de datos, transformaciones y cargas en el almacén de datos.

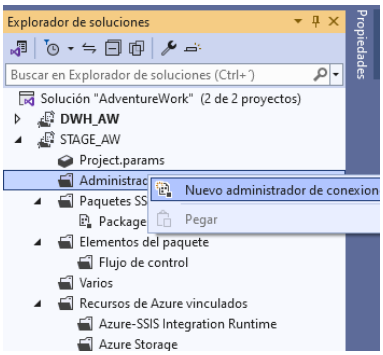


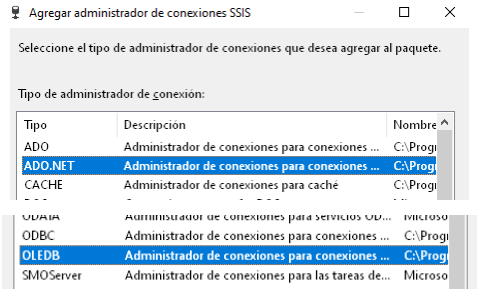
- La logica de los paquetes se divide en 2 partes:
    - **Auditoria;** La parte de Auditoría de los paquetes de ETL se enfoca en registrar y rastrear información relevante sobre el proceso de carga de datos.
        - **Recuento de datos inicial/final:** Antes y después de la transformación, se realizan recuentos de datos para asegurar que no se pierdan registros durante el proceso de ETL.
        - **Consulta del GUID para su creación:** Se consulta un identificador único (GUID) para asociarlo con cada registro nuevo que ingresa al sistema, lo que permite un seguimiento preciso de los datos a lo largo del tiempo.
        - **Inserción y/o actualización del registro**: Los datos y sus metadatos asociados, como la fecha de carga y el GUID, se insertan o actualizan en las tablas de auditoría para su posterior análisis y seguimiento.
    - **Proceso;** La parte de Proceso de los paquetes de ETL se centra en las tareas de extracción, transformación y carga de datos en el almacén de datos o el destino final.



B. Configuración de Conexiones - Global

En el apartado de administración de conexiones, se realizará las creaciones de 2 tipos de conexiones (Ado.Net y OleDB)



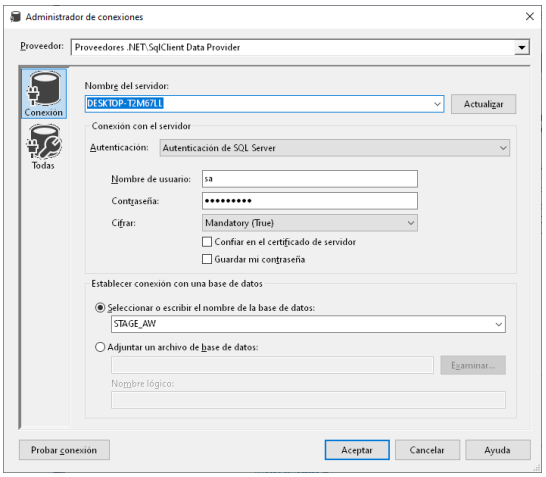


- Seleccionamos el tipo de conexión y crear nueva conexión.
- En esta nueva ventana colocaremos el nombre o ip del servidor (base de datos), las credenciales (es recomendable crear credenciales única y exclusivamente para el control y/o gestión del proceso ETL)
- Finalmente seleccionamos el nombre de la base de datos dependiendo del proyecto

Proyecto	Conexion
DWH_AW	• Ado.Net – DWH_AW • OleDB - DWH_AW
STAGE_AW	• Ado.Net – STAGE_AW

	• Ado.Net – DWH_AW • OleDB – STAGE_AW

Ejemplo Ado.Net:

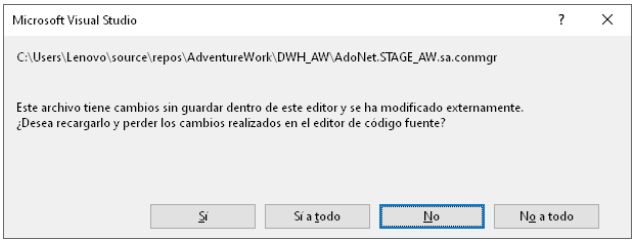


- Un dato adicional, al finalizar la creación de una conexión es recomendable renombrar esta. Teniendo en cuenta el tipo de conexión Ado.Net/OleDB, Base de datos, Usuario

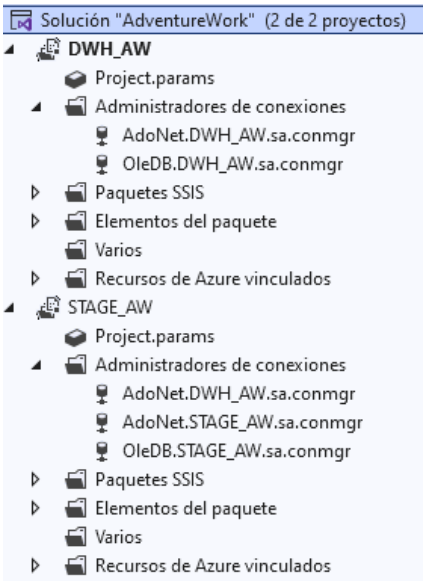
Ejemplo	Administradores de conexiones DESKTOP-T2M67LL.STAGE_AW.sa.conmgr
Proyecto DWH	Administradores de conexiones AdoNet.DWH_AW.sa.conmgr OleDB.DWH_AW.sa.conmgr
Proyecto STAGE	Administradores de conexiones AdoNet.STAGE_AW.sa.conmgr AdoNet.DWH_AW.sa.conmgr OleDB.STAGE_AW.sa.conmgr

Este proceso de creación de conexiones se debe realizar por cada proyecto y dependiendo de los requerimientos asociados.

Nota: si durante el renombre de los componentes emerge una ventana de recarga, elegir la opcion **NO.

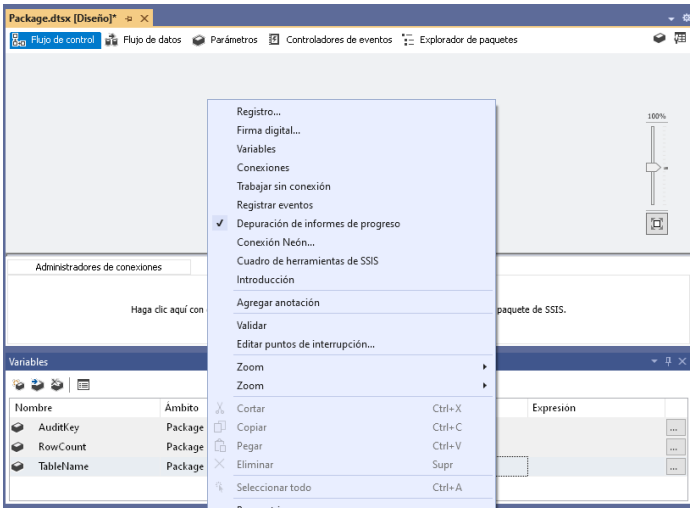


Vista final de las conexiones por proyecto



C. Configuración de componentes

**Nota: de no tener activa la pestaña de variables, seleccione el lienzo, click derecho sobre el mismo; seguidamente, seleccione variables.

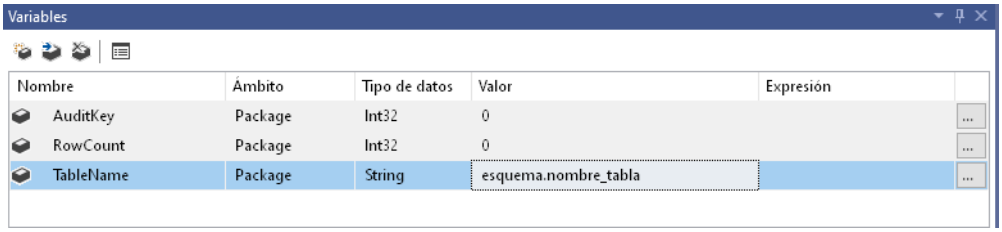


**Nota: Para acceder a las herramientas, nos ubicamos en el apartado izquierdo, cuadro de herramienta SSIS

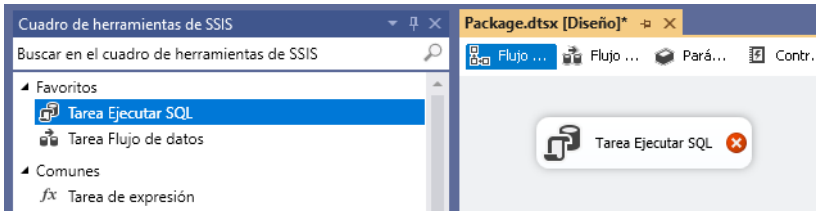


1.1. Del entorno; comenzamos creando las variables necesarias para el ciclo, las cuales son:

- AuditKey – Tipo Int
- RowCount – Tipo Int
- TableName – Tipo String – se considera el esquema y nombre de la tabla



A continuación, para la creación de tareas dinámicas usaremos el componente “tarea ejecutar sql”



Tareas/Task

1.2. Task Prior RowCnt;

- Abrimos la ventaja de la tarea y renombramos el componente



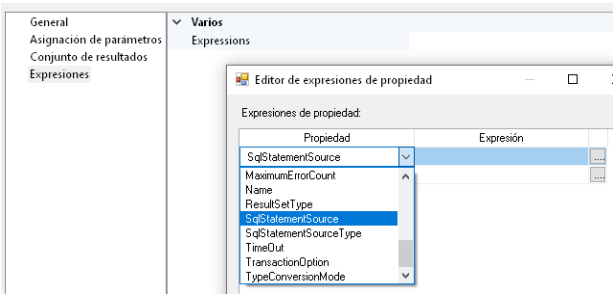
- En el apartado de conexión, seleccionaremos el tipo Ado.NET



- El conjunto de resultados será de una fila



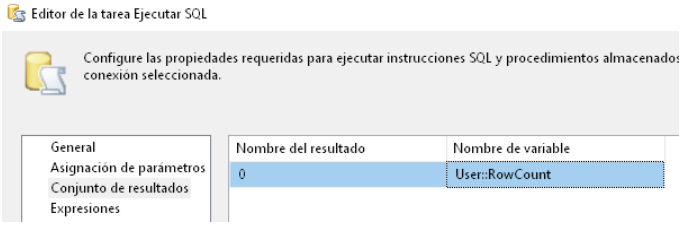
- Teniendo la preconfiguración, pasamos al apartado de expresiones; entonces, seleccionamos expresiones (…), y en la nueva ventana buscamos la propiedad SqlStatmentSource y seleccionamos (…)



Luego en la expresión colocaremos la sentencia y Ejecutamos la expresión (evaluar) y aceptar.

```
"SELECT COUNT(1) AS RowCnt FROM " +  @[User::TableName]
```

- Finalmente nos dirigimos al apartado conjunto de resultados (retorno de la expresión evaluada en la consulta anterior) y agregamos la variable que alojara el resultado, siendo en este escenario RowCount, con un valor por defecto igual a cero.



- Seleccionamos “Aceptar”, si modificamos el nombre de la tabla de la variable definida anteriormente, y accedemos al componente (o tarea) notaremos el cambio.

1.3. Task Get AuditKey

- Abrimos la ventaja de la tarea y renombramos el componente



- En el apartado de conexión, seleccionaremos el tipo Ado.NET



- El conjunto de resultados será de una fila

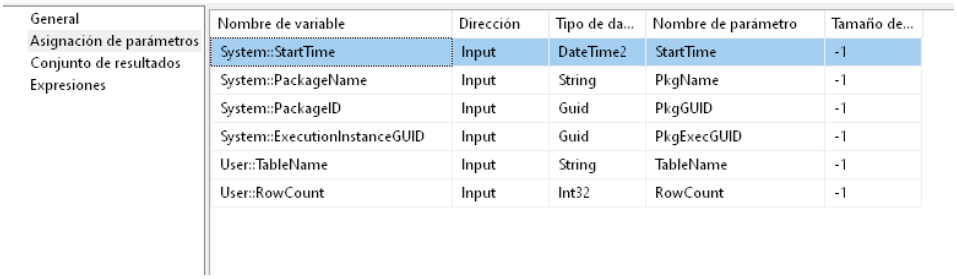


- En el apartado de SQL Statement colocaremos el siguiente codigo:

```
INSERT INTO [DIM_AUDIT]
           ([TableName]
```

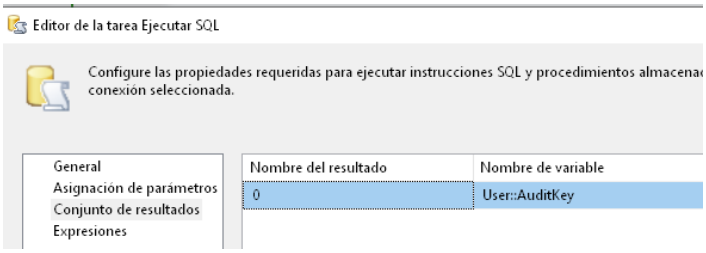
```
            ,[PkgName]
            ,[PkgGUID]
            ,[ExecStartDT]
            ,[ExecStopDT]
            ,[ExecutionInstanceGUID]
            ,[ExtractRowCnt]
            ,[TableInitialRowCnt]
            ,[TableFinalRowCnt]
            ,[SuccessfulProcessingInd])
      VALUES
            (@TableName
            ,@PkgName
            ,@PkgGUID
            ,@StartTime
            ,NULL
            ,@PkgExecGUID
            ,NULL
            ,@RowCount
            ,NULL
            ,'N')
SELECT CAST(SCOPE_IDENTITY() AS INT) AS AuditKey
go
```

- Como podrá nota el codigo hace referencia a múltiples variables, para tener el input de estas nos dirigimos al apartado asignación de parámetros y crearemos seis variables como se muestra a continuación:



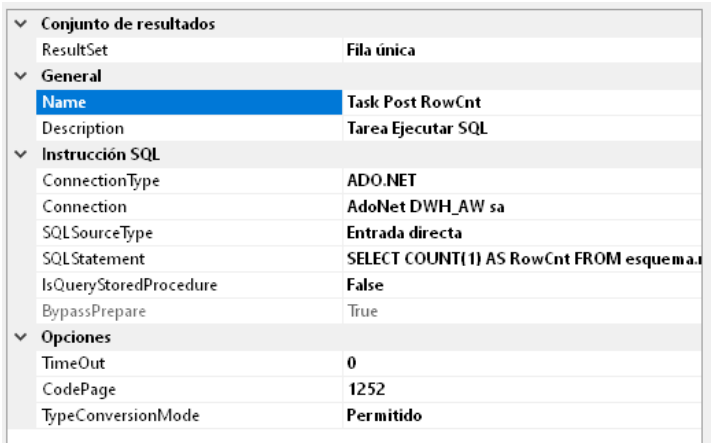
Variable	Dir.	Tipo	Nombre
System::StartTime	I	DateTime2	StartTime
System::PackageName	I	String	PkgName
System::PackageID	I	Guid	PkgGUID
System::ExecutionInstanceGUID	I	Guid	PkgExecGUID
User::TableName	I	String	TableName
User::RowCount	I	Int32	RowCount

- Finalmente, nos dirigimos al apartado de conjunto de resultados y agregamos la variable AuditKey con el valor por defecto igual a cero.



1.4. Task Post RowCnt

En este apartado, podemos obviar la creación del componente y en su lugar duplicar el ya exsistente RowCnt y solo debemos renombrarlo con el Post RowCnt. Esto es debido a que la logica sigue siendo la misma, solo cambia en la etapa de ejecucion la cual debe volver a realizar un conteo de los datos previamente insertados/actualizados/eliminados dependiendo de la logia del proceso.



1.5. Update AuditDim

- Abrimos la ventaja de la tarea y renombramos el componente



- En el apartado de conexión, seleccionaremos el tipo Ado.NET



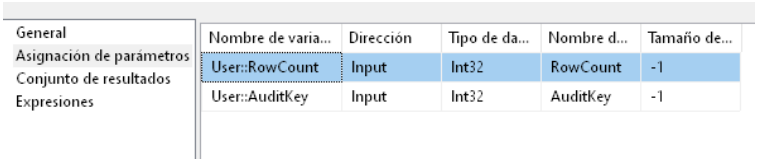
- El conjunto de resultados será Ninguno.



- En el apartado de SQL Statement colocaremos el siguiente codigo:

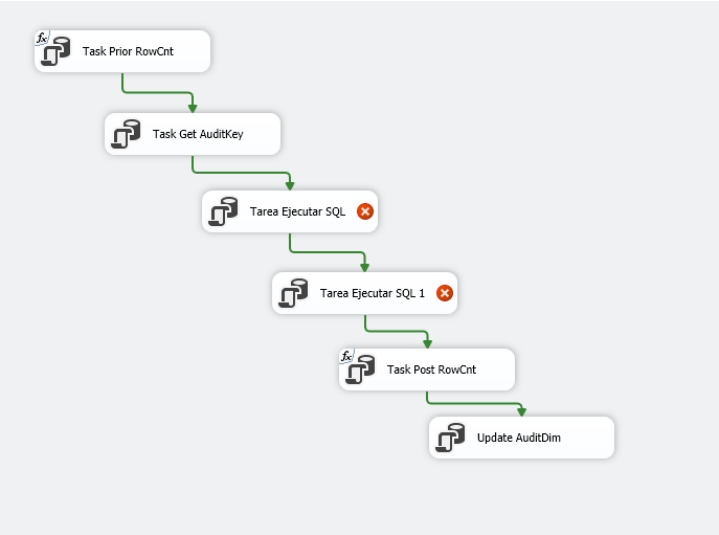
```
UPDATE [DIM_AUDIT]
SET ExecStopDT = SYSDATETIME()
 , ExtractRowCnt = @RowCount
 ,TableFinalRowCnt = @RowCount
 ,SuccessfulProcessingInd='Y'
WHERE AuditKey = @AuditKey;
```

- Por último, agregamos los parámetros correspondientes RowCount y AuditKey

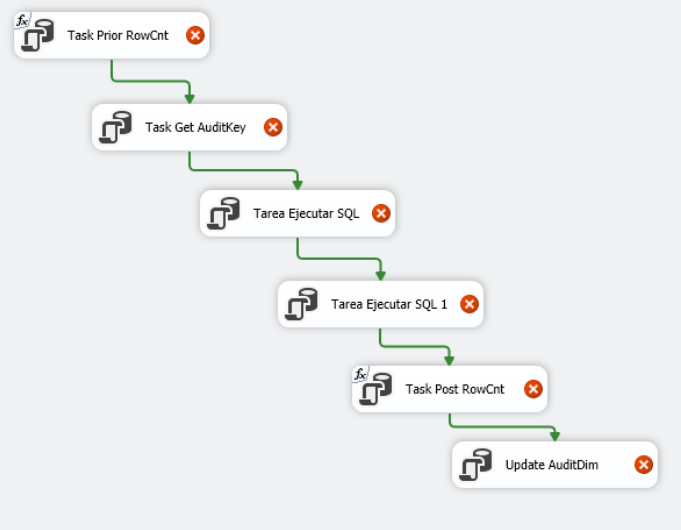


D. Consideraciones

- Podemos tener un paquete dedicado a una plantilla base con 1 o 2 tareas sin completar, esto para facilitar la creación de nuevos paquetes.



- Antes de copiar las tareas/componentes a otra paquete asegúrese de tener creada las variables dentro de cada uno, de lo contrario figurara error.



Para solucionar este problema, se deben de crear las variables y acceder a cada tarea y aceptar (con la finalidad de recargar el componente).

- Para la configuración de la conexión del componente/tarea AuditKey (get y update) tenga en cuenta que la cadena de conexión corresponde a la base de datos donde existe la tabla DIM_AUDIT, siendo este ejercicio DWH_AW.

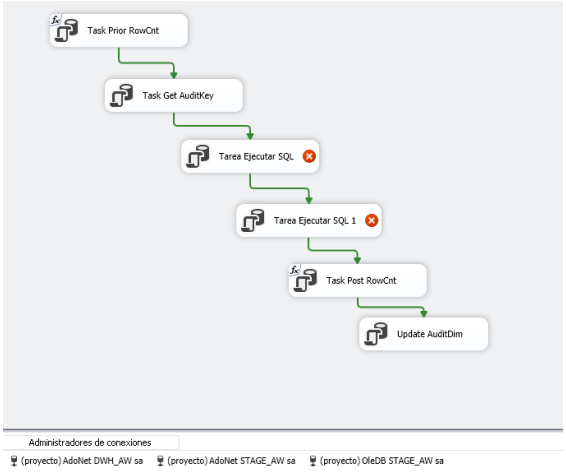
E. Paquetes

A continuación, se mostrará la creación del apartado proceso dentro del flujo de tarea para diferentes situaciones.

STAGE_AW

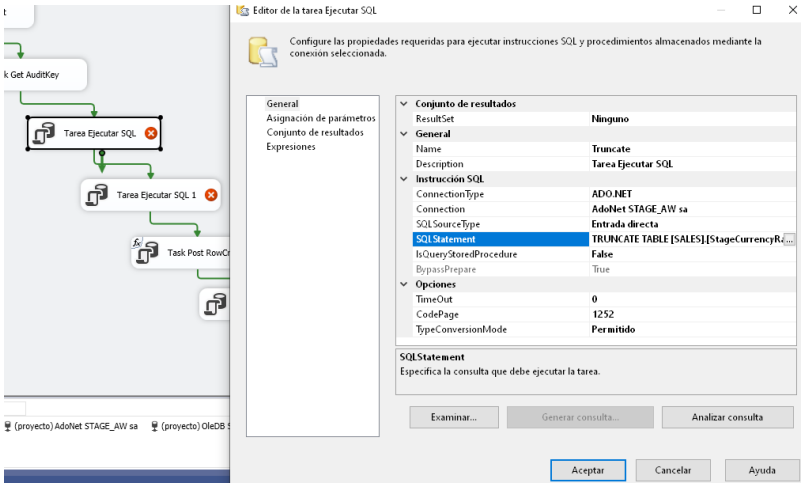
- Comenzamos creando la ejecucion de un store procedure mediante una tarea SSIS; dentro de la base de datos stage existen 7 procedimientos de los cuales solo tomaremos 1 como ejemplo.

  Iniciamos creando las variables del paquete y posteriormente copiando las tareas de la plantilla creada previamente.

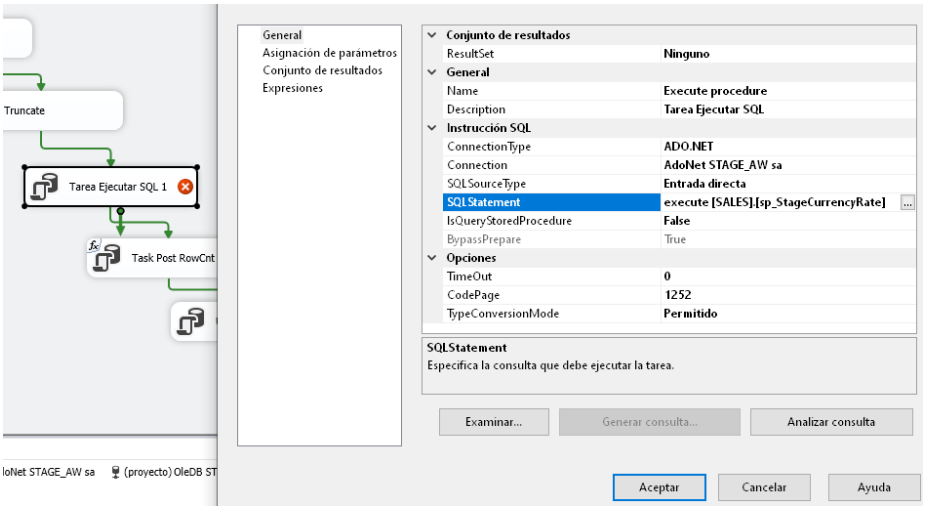




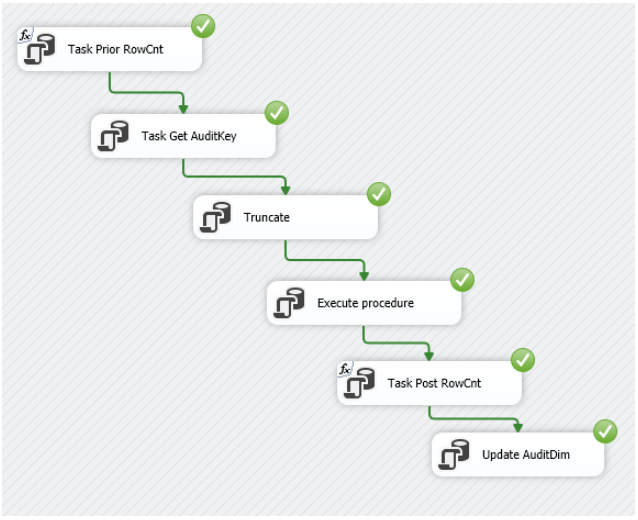
La primera tarea será equivalente a la limpieza (Truncate) de la tabla, tenga en cuenta que esta tarea no es automática (como el caso de otras tareas) debido a que no siempre la limpieza es a una única tabla, tenga en cuenta que estas tareas son única y exclusivamente orientadas a una logica (proceso).



La segunda tarea es derivada al query execute correspondiente al proceso (dado que la logica existe dentro de este).



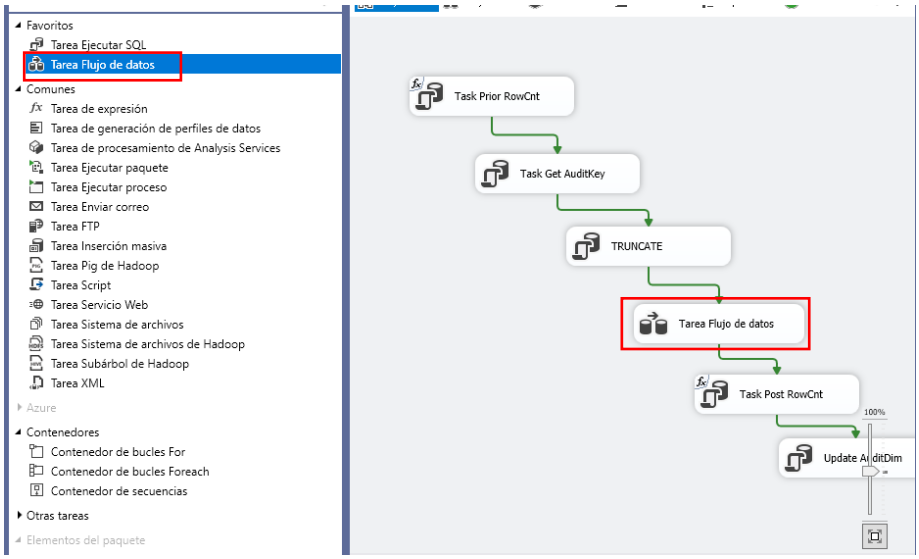
Finalmente, tenemos el primer paquete completado. Tenga en consideración que el nombre de este paquete debe hacer referencia a la logica interna (puede ser el nombre de la tabla destino).



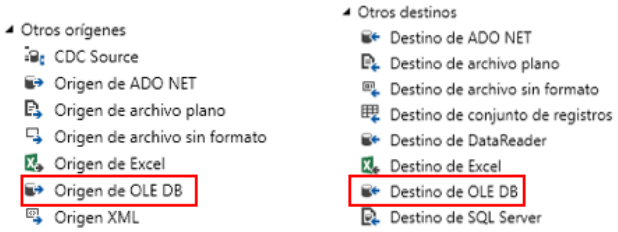
Este proceso se debe repetir por cada store que exista en la base de datos STAGE.

- Luego de completar el primer conjunto de paquetes orientados a los procedimientos, notaremos que existen 2 tablas las cuales no se cargan de esta forma concretamente StageProduct y StageProductCategory en el caso de la base de datos STAGE_AW. Para ello, usaremos una carga mediante conexión entre base de datos.

  Iniciamos configurando la primera tarea como truncate de la tabla correspondiente y luego reemplazamos la siguiente tarea de sql por una de flujo de datos



Si damos doble click sobre la tarea/componente se nos apertura una nueva ventana, y adicionalmente el cuadro de herramientas se renueva; de este nuevo cuadro de herramientas nos ubicamos en el apartado de origentes/destinos, donde utilizaremos los componentes que terminan en OLE DB para el origen y el destino.

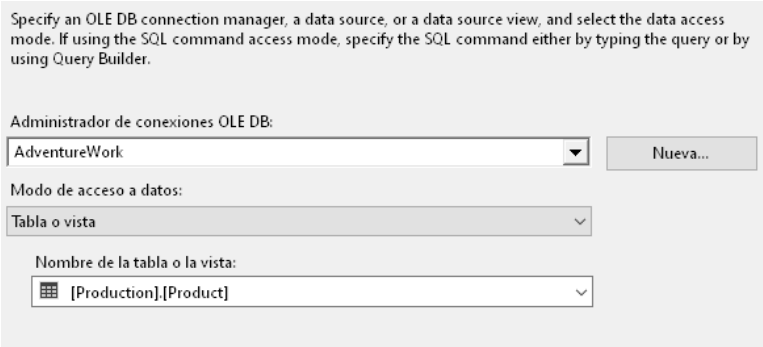




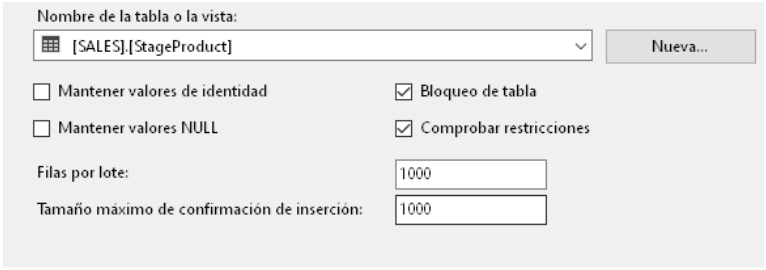
Si bien, tenemos componentes para realizar tratamientos de los datos, no son necesarios para este escenario dado que la estructura de datos es la misma.

Otro punto por abordar es la no existencia de la conexión a la base de datos origen (en este escenario adventure work), por ello seleccionaremos en nueva y crearemos una conexión de tipo OleDB orientado a este origen; el motivo que esta no sea de tipo general es debido a su carente uso (solo en 2 escenarios).

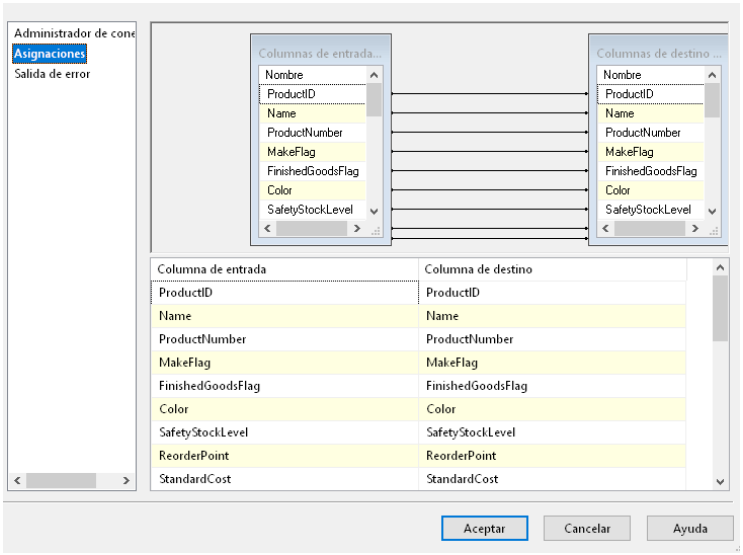
Luego de crear la conexión local en el paquete buscamos la tabla correspondiente que es igual a Production.Product



En el destino, seleccionaremos la conexión y tabla correspondiente (StageProducto/StageProductCategory). Notara que existe un apartado que limita la cantidad de filas a insertar, siendo una buena práctica aplicar un límite y no dejar los valores por defecto.



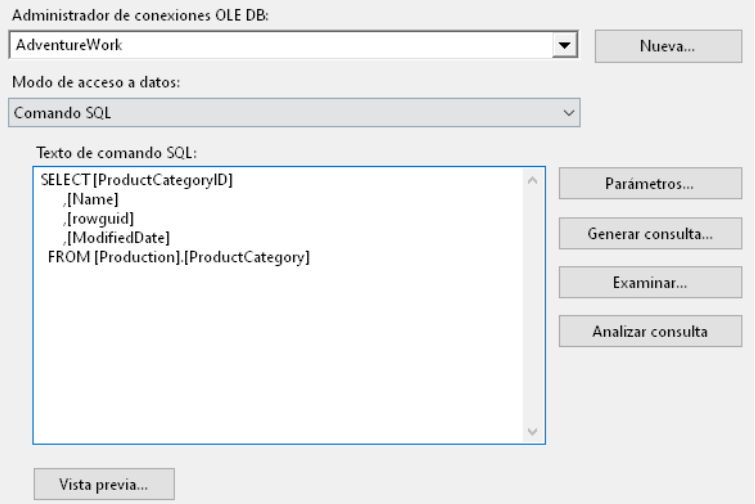
Finalmente, accedemos al apartado asignaciones el cual mapeará automáticamente todas las columnas coincidentes entre origen y destino, caso contrario las columnas sean diferentes se deberá hacer manualmente.



**En el escenario de productCategory podemos usar una logica diferente, la cual consistiría en aplicar una consulta sql.

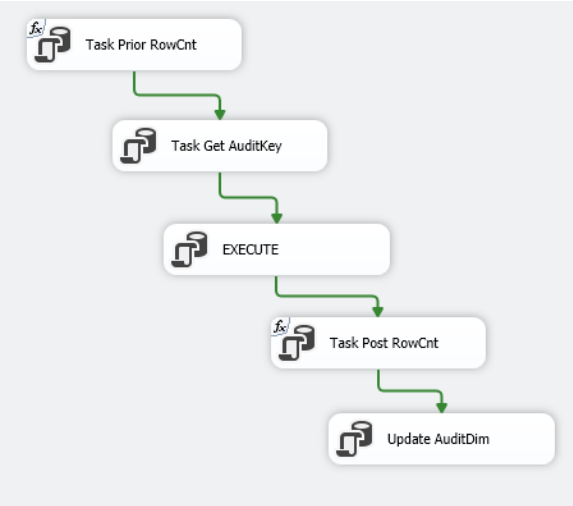
```
SELECT [ProductCategoryID]
              ,[Name]
              ,[rowguid]
```

```
              ,[ModifiedDate]
  FROM [Production].[ProductCategory]
```



DWH_AW

- Acotando sobre la creación de los paquetes referentes a las dimensiones, estas no deben tener una tarea de tipo “truncate”, por lo que al momento de la creación de los paquetes obviaremos 1 componente/tarea



- En el proyecto asociado al DWH tendremos dos escenarios, siendo el primero la no existencia de un origen para la tabla DimFecha, y un segundo escenario que compete a la tabla FactSales (e indirectamente a ErrorFactSales).
- En el primer escenario se contempla la carga de ficheros desde una carpeta (la cual puede ser manipulada por algún usuario), para este escenario se creará una tabla especializada la cual albergará el origen de esta carpeta u ficheros.

```
CREATE TABLE PATH_FILE(
ID INT IDENTITY(1,1),
NEGOCIO NVARCHAR(50),
RUTA NVARCHAR(500) ,
TableName NVARCHAR(500))
```

Results | Messages

	ID	NEGOCIO	RUTA	TableName
1	1	SALES	C:\Users\Lenovo\Desktop\Ejercicio analitica\Dim_Fecha	[GENERICO].[DimFecha]

Luego de configurar el paquete (crear las variables y copiar los componentes de la plantilla), se adicionarán 2 variables las cuales son “archivo” y “fecha” de tipo string e integer respectivamente.

Variables

Nombre	Ámbito	Tipo de datos	Valor
archivo	Package2	String	ficheros
AuditKey	Package2	Int32	0
fecha	Package2	Int32	0
RowCount	Package2	Int32	0
TableName	Package2	String	[GENERICO].[DimFecha]

A continuación, la logica empleara 2 componentes adicionales:

- Contenedores
  - Contenedor de bucles For
  - Contenedor de bucles Foreach
  - Contenedor de secuencias

Contenedor de secuencias

Un contenedor de secuencias es un objeto que agrupa y controla la ejecución de un conjunto de tareas relacionadas. Estos contenedores permiten organizar y administrar las tareas de flujo de control de manera más efectiva.

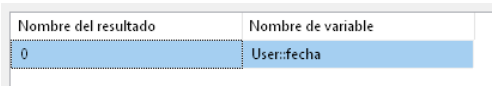


Para el componente/Tarea FechaMax utilizaremos una sentencia sql para obtener la última fecha.

```
SELECT ISNULL(MAX([FechaKey]), -1)   FROM DWH_AW].[GENERICO].[DimFecha]
```

Conjunto de resultados	
ResultSet	**Fila única**
General	
Name	**FechaMax**
Description	**Tarea Ejecutar SQL**
Instrucción SQL	
ConnectionType	**ADO.NET**
Connection	**AdoNet DWH_AW sa**
SQLSourceType	**Entrada directa**
SQLStatement	**SELECT ISNULL(MAX([FechaKey]), -1) FROM [D**
IsQueryStoredProcedure	**False**
BypassPrepare	True
Opciones	
TimeOut	**0**
CodePage	**1252**
TypeConversionMode	**Permitido**

Esta tarea tendrá como salida una fila única la cual será almacenada en la variable “fecha”.

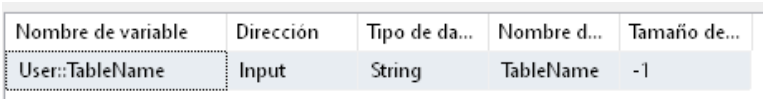


Nombre del resultado	Nombre de variable
0	User::fecha

Adicionalmente, configuraremos el componente “Ruta” el cual será el encargado de retornar la cadena origen de los ficheros o rutas dependiendo de la logica. En el caso de tener más de 1 cadena orientara a una misma tabla (en este escenario dim_fecha) se recomienda usar query específicos y no dinámicos.

```
        SELECT RUTA FROM [dbo].[PATH_FILE]
  WHERE [TableName] = TableName
```

Luego de configurar el componente, asignaremos el parámetro “TableName”.



Nombre de variable	Dirección	Tipo de da...	Nombre d...	Tamaño de...
User::TableName	Input	String	TableName	-1

Finalmente, seleccionamos la variable la cual albergará el resultado.



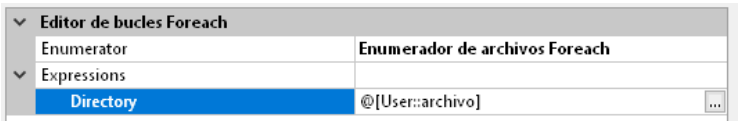
Nombre del resultado	Nombre de variable
0	User::archivo

Contenedor de Bucles Foreach

Permite iterar sobre una colección de elementos y ejecutar un flujo de control para cada uno de ellos. Esto es particularmente útil cuando necesitas procesar múltiples archivos, tablas, o cualquier otro conjunto de elementos de manera similar.



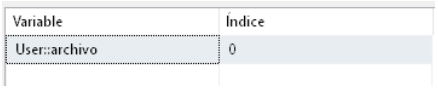
Iniciamos dando doble click al componente, y nos dirigimos a la sección “colección”; en este apartado encontraremos una sección denominada editor de bucles Foreach en donde escogeremos la opcion “Enumerador de archivos Foreach”, tenga en cuenta que se dispone de una amplia gama de opciones para diferentes situaciones o necesidades. Luego, en el apartado “Expressions” agregaremos el tipo “Directory” permitiendo al componente la dinamización de este parámetro, proviniendo de la variable de usuario “archivo”.



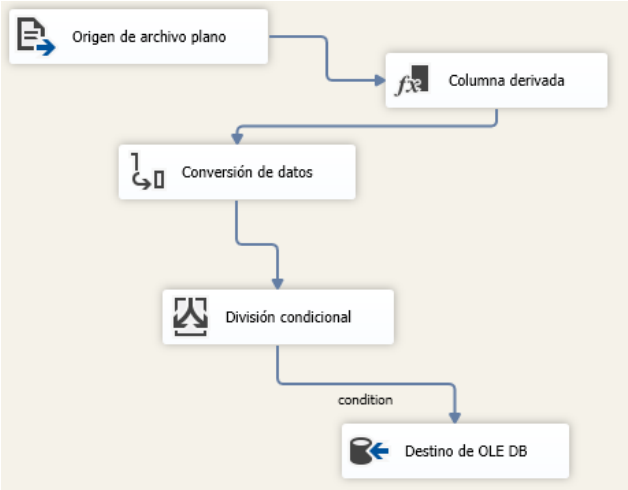
Una vez configurado el encabezado, notaremos que en la sección carpeta figura el valor por defecto de la variable (contenido que previamente configuramos); nos enfocaremos en el apartado archivos, en donde podemos limitar el tipo de ficheros buscados por el componente (para este escenario archivos .csv) siendo "*" cualquier valor, esto es fundamental para escenarios donde tengamos un único origen y diferentes tipos de ficheros en el mismo, pudiendo agregar texto representativo para seccionar los mismos (por ejemplo xxxx_ventas.csv, xxxx_rrhh.xlxs. xxxx_admin.json). También podemos configurar que parte del archivo se va a recuperar (ruta completa, nombre.extensión y solo el nombre del archivo) y el recorrido de posibles subcarpetas.



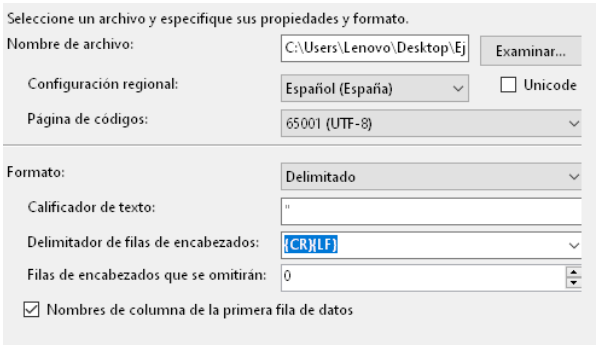
Finalmente, agregamos la variable de salida en el apartado de asignaciones de variables, en donde podrá notar que reusamos la variable que contiene el directorio o ruta, esto se debe a que el componente ya almacena el valor de este dejando libre la variable (permitiéndose limitar la creación de variables innecesarias).



Ahora se puede configurar el componente "Tarea de flujo de datos" del contendor "bucles Foreach".



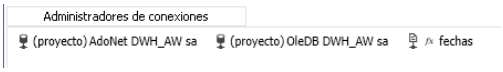
Comenzamos agregando un componente de tipo origen de archivo plano y al abrirlo seleccionaremos en “new”, este componente solicita una ruta previamente a su automatización debió a la previa configuración de las columnas. Luego seleccionamos una configuración regional que respete el codigo asociado al texto del documento (para esta plantilla será UTF-8) y el calificador de texto será de tipo comillas dobles.



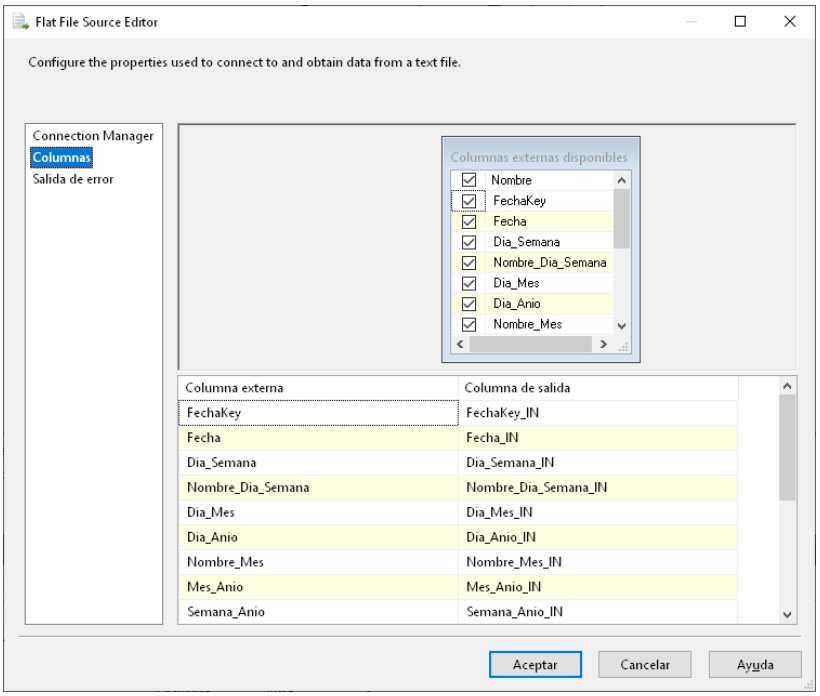
En la sección de columnas, podemos definir el tipo de separación que se empleara sobre los datos, generalmente el componente lo hace automáticamente y en casos excepciones suele fallar.

En la sección de opciones avanzadas podremos realizar ajustes y preconfiguraciones adicionales, generalmente solo se modifica el ancho de la columna y posteriormente se agregan pasos especializados para su tratamiento.

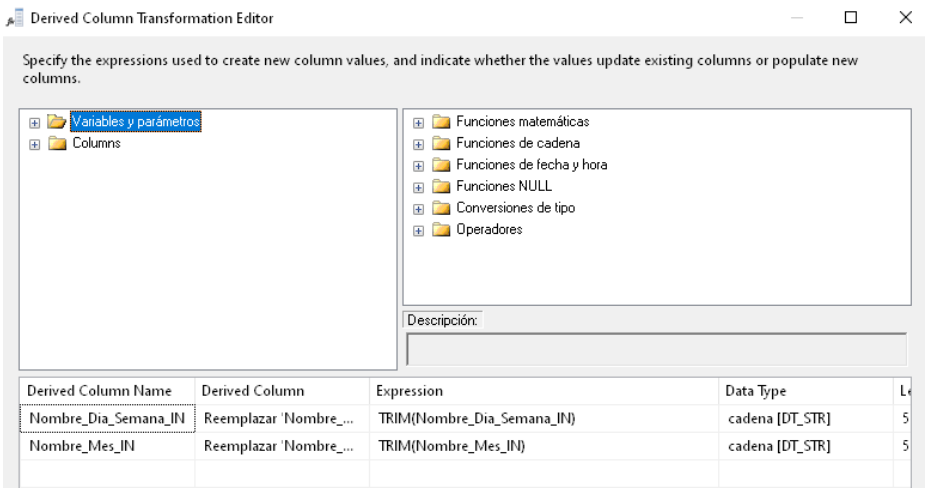
Por último, revisamos los datos en la opcion de vista previo y aceptamos. Notara que en el apartado de conexiones se ha creado este conector.



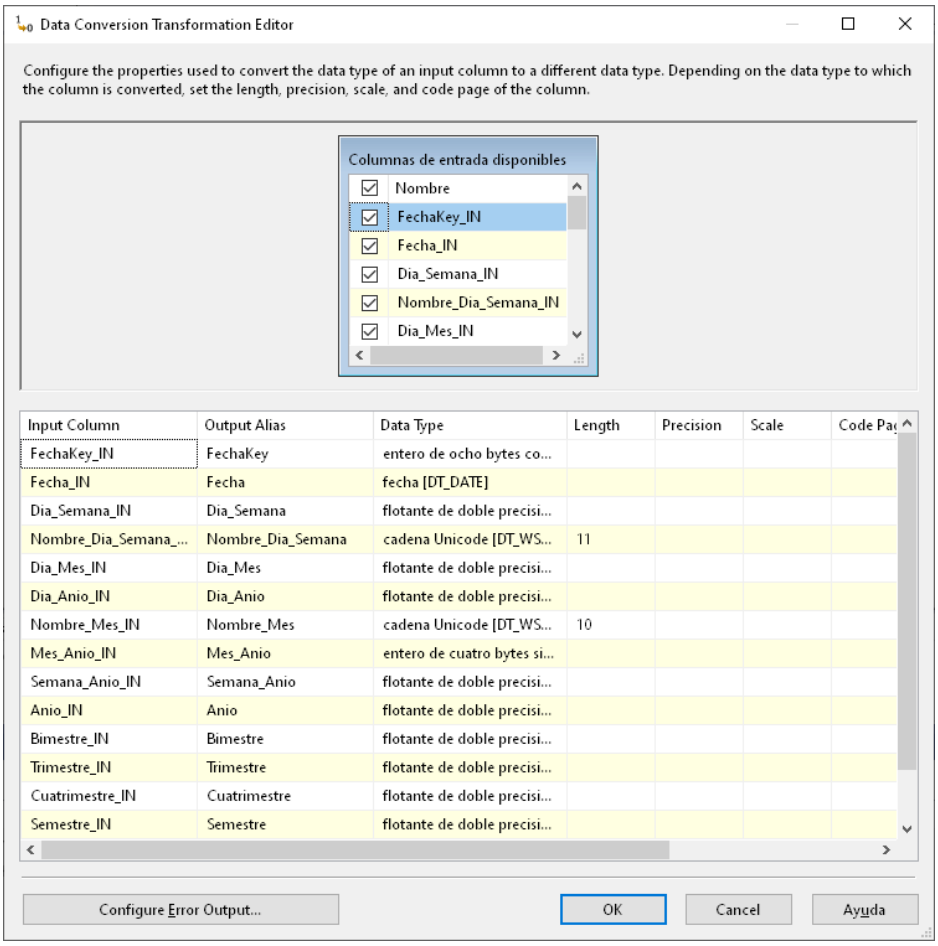
Luego, volvemos a acceder al componente origen de archivo plano y nos iremos al apartado columnas en donde agregaremos “_IN” a las columnas de salida.



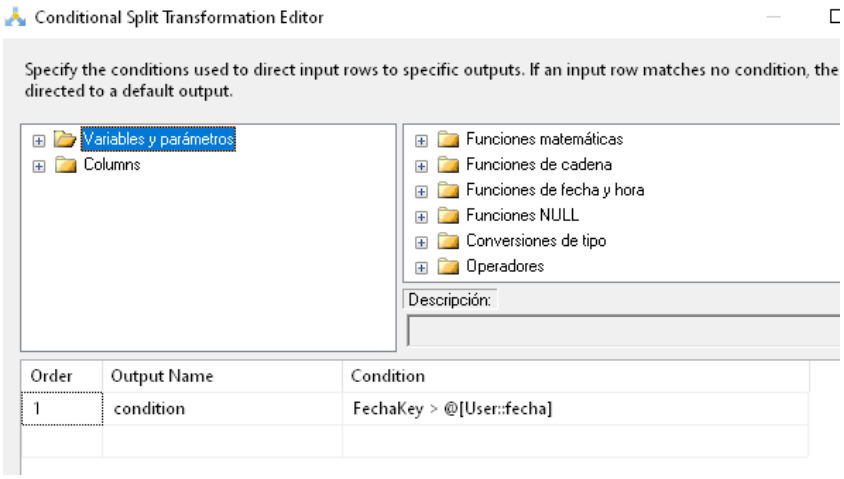
Teniendo configurado el componente de origen de datos procedemos a agregar el componente “columna derivada” dentro del apartado “comunes”, dentro de este componente podemos realizar diversas transformaciones que derivan en reemplazos o generación de nuevas columnas; para este escenario, usaremos un TRIM para eliminar posibles espacios en las columnas Nombre_Dia_Semana_In y Nombre_Mes_in, y posteriormente el reemplazo de esta.



Luego procedemos a agregar el componente “conversión de datos” en donde realizaremos las transformaciones de tipo de dato correspondiente a la tabla destino (DIM_FECHA), y adicionalmente renombraremos las columnas (quitaremos el _IN y cualquier valor autogenerado por el mismo componente).

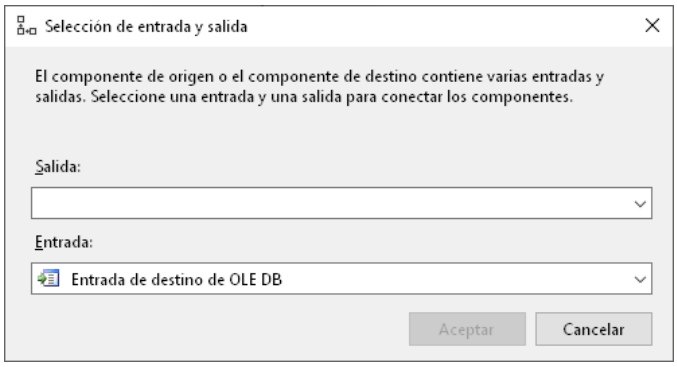


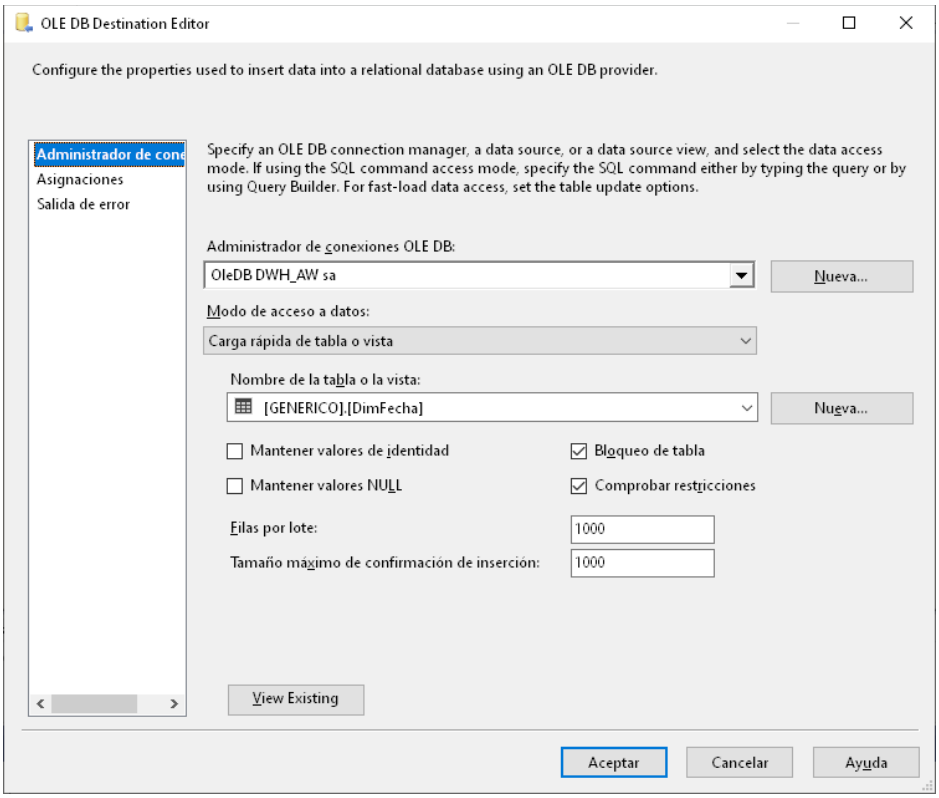
Teniendo el ciclo de transformación y configuración de los componentes y datos procedemos a generar un filtro mediante el componente “División Condicional”, el cual se basa en la simulación de volcado incremental teniendo como base la columna “FechaKey” (la cual según la logica de la tabla debe ser única), este componente retorna 2 conjunto de datos derivados de la expresión logica.



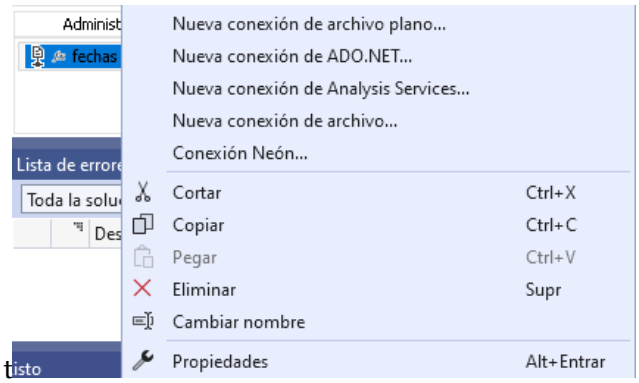
Por último, agregamos el componente OLEDB de tipo destino apuntando a la tabla GENERICO.DIM_FECHA; adicionalmente, notará que al momento de realizar el

enrutamiento emergerá una venta la cual solicitando el conjunto de datos a seguir el flujo (para esta situación será “condition”)

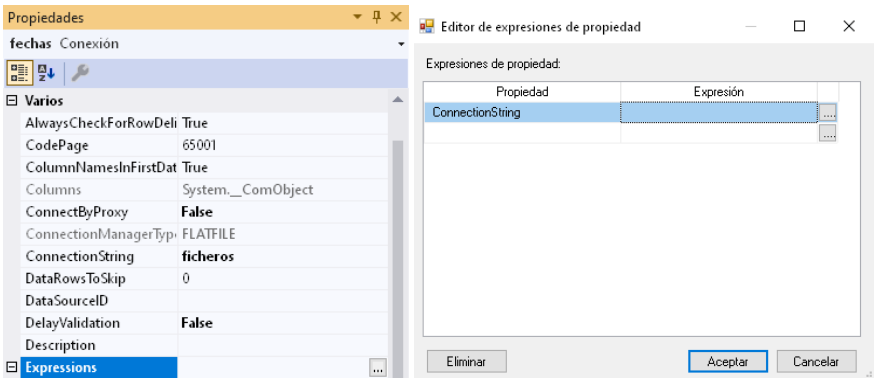


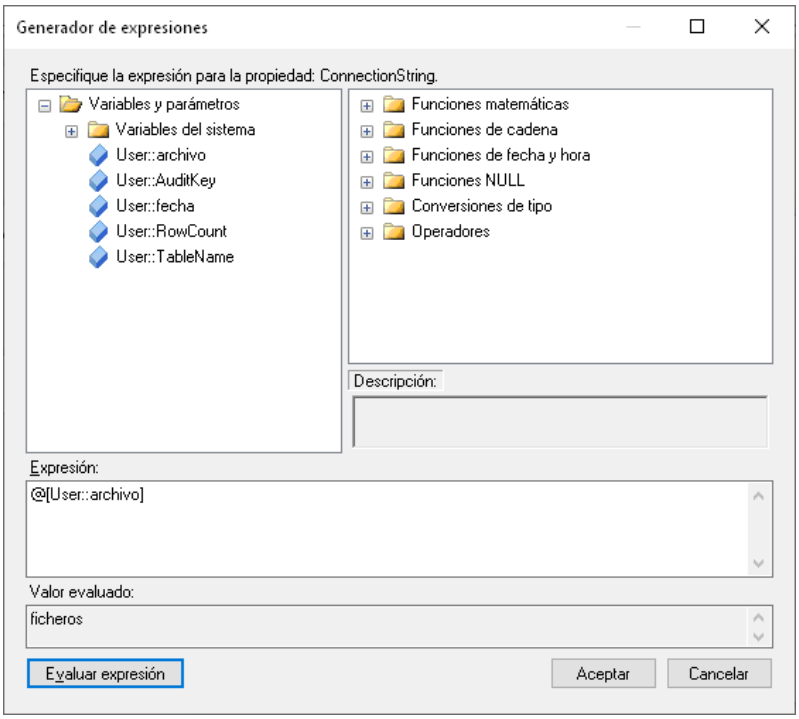


Finalmente, configuramos la expresión de la conexión del “origen de archivo plano” accediendo al apartado propiedades.

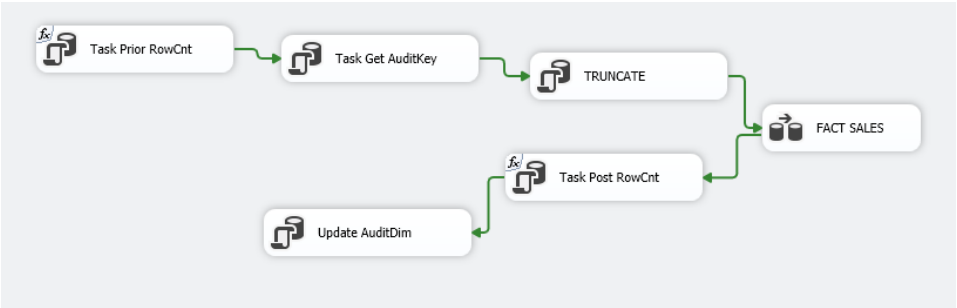


Luego, en el apartado de propiedades nos dirigimos a la opcion “Expressions” y creamos la propiedad “ConnectionString”, al cual se le debe asignar la variable de usuario “archivo”.



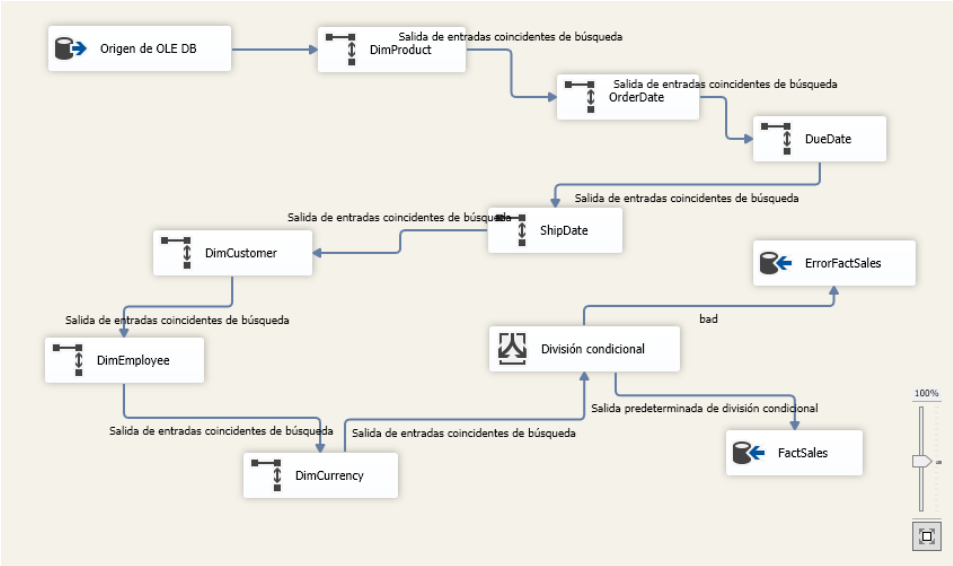


- En el segundo escenario, contemplando la tabla FACTSALE (teniendo en cuenta la existencia de un store para su carga) usaremos los diferentes componentes proporcionados por Integration Services.



Como puede observar, la distribución de tareas es similar a las cargas realizadas en el stage siendo las 2 tareas del proceso (logica), el truncar la tabla factsale y

posteriormente el uso de las herramientas SSIS; esto significara la réplica del store factsale usando las propias herramientas SSIS como se aprecia en la imagen.



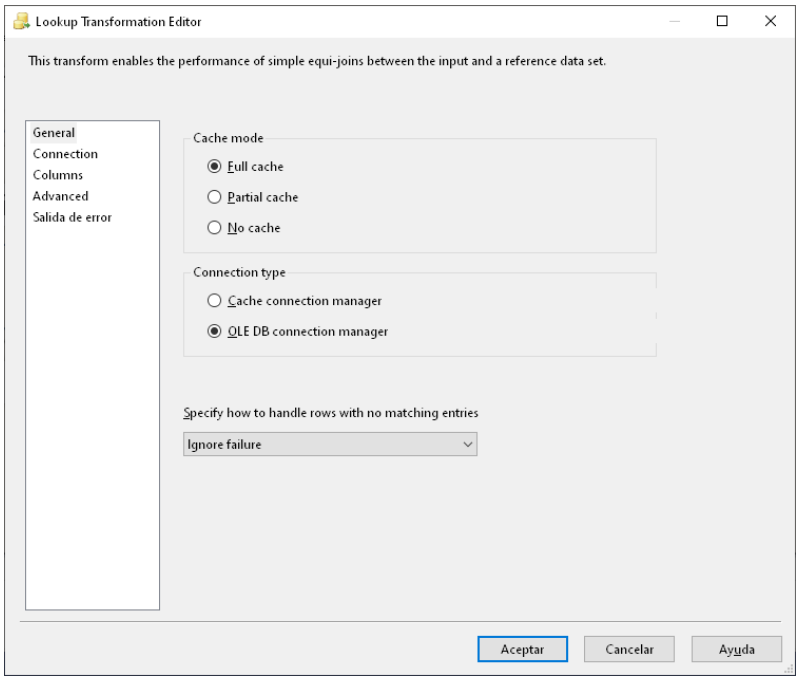
Origen OLE DB

```
SELECT d.[SalesOrderID]
      ,d.[SalesOrderDetailID]
      ,d.[ProductID]
      ,d.[SpecialOfferID]
      ,d.[OrderQty]
      ,d.[UnitPrice]
      ,d.[UnitPriceDiscount]
          ,o.[RevisionNumber]
      ,o.[SalesOrderNumber]
      ,o.[PurchaseOrderNumber]
      ,o.[AccountNumber]
      ,o.[OrderDate]
      ,o.[DueDate]
      ,o.[ShipDate]
      ,o.[CustomerID]
      ,o.[EmployeeID]
      ,o.[CurrencyRateID]
      ,o.[Status]
      ,o.[OnlineOrderFlag]
      ,o.[SubTotal]
      ,o.[TaxAmt]
      ,o.[Freight]
  FROM [Stage_AW].[SALES].[StageSalesOrderDetail] d
  inner join [Stage_AW].[SALES].[StageSalesOrderHeader] o
                        on d.SalesOrderID = o.SalesOrderID
```

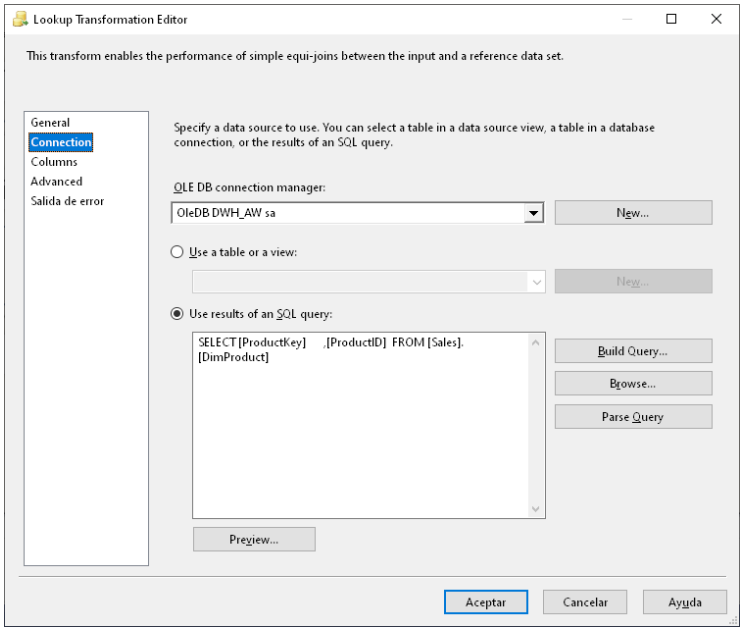
El esquema representa el uso del componente “búsqueda” para relacionar las diferentes dimensiones, un proceso escalonado y ordenado; luego de juntar las dimensiones con sus respectivas columnas (join’s) y llevar la(s) columnas correspondientes se procede a utilizar el componente “división condicional” el cual evalúa una serie de condicionales dando como resultado la fragmentación de dataset, siendo un conjunto los que se evalúan como TRUE y otro como FALSE; estos resultados se almacenan en 2 tablas diferentes “errorfactsales” y “factsales”.

La configuración de del objeto búsqueda consiste en ajustar los parámetros del apartado general en modo “full cache”, hacer uso del tipo de

conexión OLE DB y omitir posibles fallos en las entradas, tenga en cuenta que esta última configuración puede repercutir en pérdidas de datos.

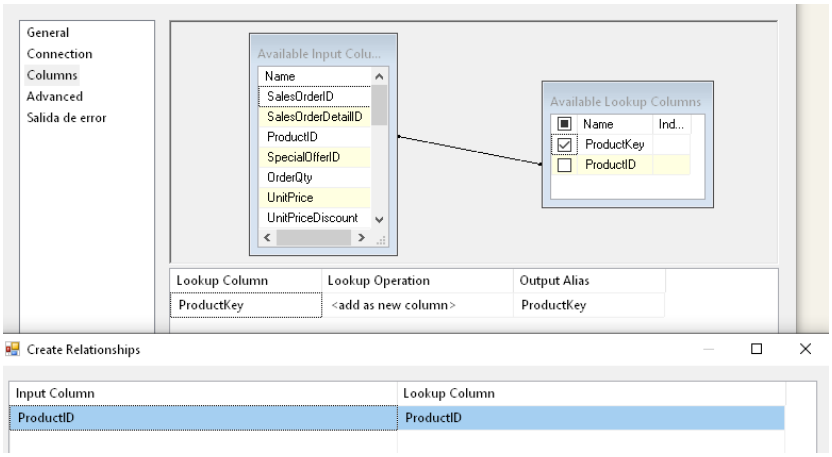


El apartado de conexión es una pantalla intuitiva la cual consta de la selección de una tabla/vista origen (join) o, el uso de SQL.

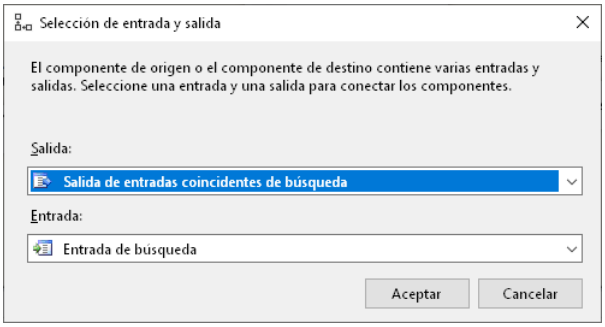


El apartado columnas se divide en 2 operaciones; la primera, consiste en la unión de las tablas involucradas mediante su columna correspondiente pudiendo arrastrar la misma sobre su equivalente o seleccionando el apartado en blanco y usar la opcion de “editar asignaciones”, la segunda, consiste en la selección de las columnas que se unirán a la tabla principal y la asignación del alias correspondiente a la misma, tenga en cuenta que el nombre de las columnas dentro de una tabla (caso sql) debe ser única.

Como puede apreciar en la figura, se unen las tablas involucradas a través de la columna ProductID y solo se lleva la columna “ProductKey” utilizando como alias su nombre. En caso de que la adicción de la columna no aplique y se busque el reemplazo de una columna sobre la tabla principal, se debe cambiar la opcion central “lookup operation”.



Tenga en cuenta que al relacionar el componente búsqueda se solicitara la selección del tipo de salida, el cual es “Salida de entradas coincidentes de búsqueda”



A continuación, se presenta las asignaciones correspondientes a cada proceso dentro del componente “búsqueda”

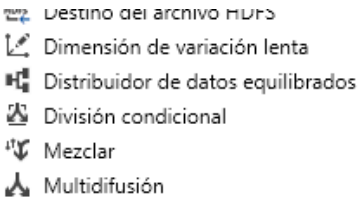
Componente	SQL	Join Left	Joing Rigth	Lookup operation	Lookup column	Output Alias
DimProduct	SELECT [ProductKey],[Product ID] FROM [Sales].[DimProduct]	ProductID	ProductID	add	ProductKey	ProductKey
OrderDate	SELECT [FechaKey],[Fecha] FROM [GENERICO].[DimFecha]	OrderDate	Fecha	add	FechaKey	OrderDateKey
DueDate	SELECT [FechaKey],[Fecha] FROM [GENERICO].[DimFecha]	DueDate	Fecha	add	FechaKey	DueDateKey
ShipDate	SELECT [FechaKey],[Fecha]	ShipDate	Fecha	add	FechaKey	ShipDateKey

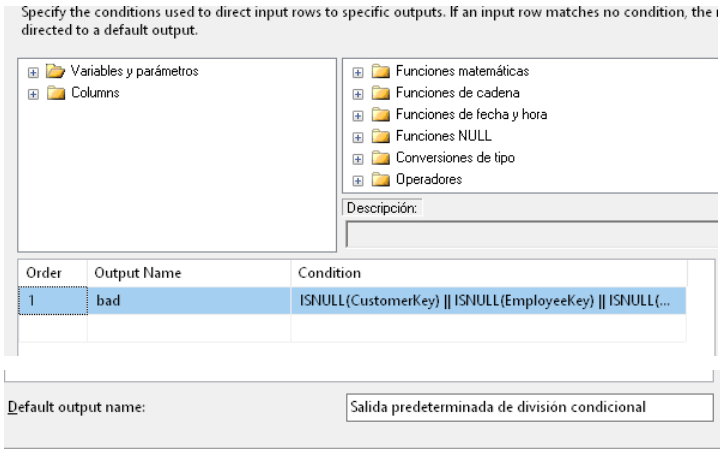
	FROM [GENERICO].[DimFecha]					
DimCustomer	SELECT [CustomerKey],[CustomerID] FROM [Sales].[DimCustomer]	CustomerID	CustomerID	add	CustomerKey	CustomerKey
DimEmployee	SELECT [EmployeeKey],[EmployeeID] FROM [Sales].[DimEmployee]	EmployeeID	EmployeeID	add	EmployeeKey	EmployeeKey
DimCurrency	SELECT [CurrencyKey],[CurrencyRateID]FROM [Sales].[DimCurrency]	CurrencyRateID	CurrencyRateID	add	CurrencyKey	CurrencyKey

Luego de haber realizado el flujo de tipo join mediante el componente "Búsqueda", se procede a incorporar una logica la cual consiste en fragmentar el dataset en aquellos que sus llaves son de tipo null como se presente en el siguiente codigo:

```
ISNULL(CustomerKey) || ISNULL(EmployeeKey) || ISNULL(CurrencyKey) ||
ISNULL(ProductKey) || ISNULL(OrderDateKey) || ISNULL(DueDateKey) ||
ISNULL(ShipDateKey)
```

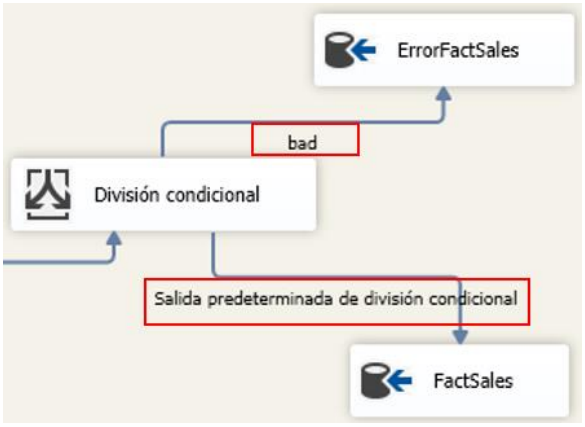
Esta logica parte desde la inclusión del componente "División condicional" en el cual se considera la condición del particionamiento del conjunto de datos (o evaluación del mismo) en base a la logica ya mencionada y a esta condicional se le asigna un nombre, tenga en cuenta que puede encadenar más de una condicional teniendo el cuenta el orden de aplicación de estas.



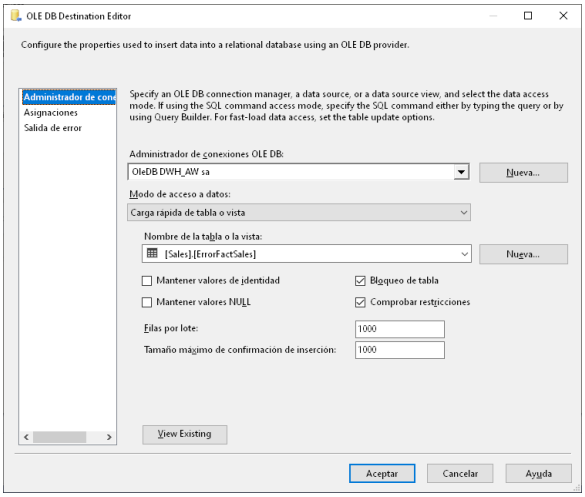


Finalmente creamos los dos componentes que insertaran las filas en sus correspondientes tablas (destino), al momento de relacionar el componente "división condicional" emergerá una ventana la cual solicitara que dataset viajara

a través de ese flujo; “Bad” corresponde al alias asignado a la condicional el cual poblara la tabla “ErrorFactSales” y el parámetro por defecto “Salida predeterminada de división condicional” (siendo los que cumplen la logica) poblara la tabla “FactSales”.



Se muestra una referencia del editor OLEDB de tipo destino el cual ya se ha ido explorando previamente, tenga en cuenta que luego de configurar el apartado de “administrador de conexiones” debe dirigirse al apartado “asignaciones” dado que sin este paso el sistema no puede mapear las columnas; también, debe tener en cuenta que solo se mapean automáticamente las columnas con coincidentes.



Antes de ejecutar las tablas de hechos, es crucial abordar la inclusión de comodines dentro de las dimensiones. Esto nos permite identificar y prevenir posibles errores. Por ejemplo, incluir un valor predeterminado como -1:

```
SELECT TOP (1000) [CurrencyKey]
      ,[CurrencyRateID]
      ,[CurrencyRateDate]
      ,[FromCurrencyCode]
      ,[ToCurrencyCode]
  FROM [Demo_DMT_AW].[Sales].[DimCurrency]
```

Messages

CurrencyKey	CurrencyRateID	CurrencyRateDate	FromCurrencyCode	ToCurrencyCode
-1	-1	2021-02-11 21:33:43.870	SiD	SiD
1	143	2011-06-10 00:00:00.000	USD	VEB
2	147	2011-06-11 00:00:00.000	USD	CAD

```
SET IDENTITY_INSERT [Sales].[DimCurrency] ON;
INSERT INTO [Sales].[DimCurrency]
           ([CurrencyKey]
                  ,[CurrencyRateID]
           ,[CurrencyRateDate]
           ,[FromCurrencyCode]
           ,[ToCurrencyCode])
     VALUES
           (-1
                  ,-1
           ,'2021-02-11'
           ,'SiD'
           ,'SiD'
                  )
SET IDENTITY_INSERT [Sales].[DimCurrency] OFF;
GO
```

```
SET IDENTITY_INSERT [Sales].[DimCustomer] ON;
INSERT INTO [Sales].[DimCustomer]
           ([CustomerKey]
                        ,[CustomerID]
                        ,[AccountNumber]
                        ,[AddressID]
                        ,[FirstName]
                        ,[MiddleName]
                        ,[LastName]
                        ,[EmailAddress]
                        ,[PhoneNumber]
                        ,[AddressLine1]
                        ,[AddressLine2]
                        ,[City]
                        ,[PostalCode]
                        ,[StateProvinceCode]
                        ,[StateProvinceName]
                        ,[CountryRegionCode]
                        ,[CountryRegionName])
     VALUES
           (-1,
                  -1,
                  'Sin Datos',
                  -1,
                  'Sin Datos',
                  'Sin Datos',
                  'Sin Datos',
                  'Sin Datos',
                  'Sin Datos',
                  'Sin Datos',
                  'Sin Datos',
                  'Sin Datos',
                  'Sin Datos',
                  'SiD',
                  'Sin Datos',
                  'SiD',
                  NULL
                  )
SET IDENTITY_INSERT [Sales].[DimCustomer] OFF;
GO
```

```
SET IDENTITY_INSERT [Sales].[DimEmployee] ON;
INSERT INTO [Sales].[DimEmployee]
           ([EmployeeKey]
                           ,[EmployeeID]
                           ,[NationalIDNumber]
                           ,[FirstName]
                           ,[MiddleName]
                           ,[LastName]
                           ,[LoginID]
                           ,[JobTitle]
                           ,[BirthDate]
                           ,[MaritalStatus]
                           ,[Gender]
                           ,[HireDate]
                           ,[SalariedFlag]
                           ,[CurrentFlag]
                           ,[EmailAddress]
                           ,[PhoneNumber])
     VALUES
           (-1
                     ,     -1
                     ,     'Sin Datos'
                     ,     'Sin Datos'
                     ,     'Sin Datos'
                     ,     'Sin Datos'
                     ,     'Sin Datos'
                     ,     'Sin Datos'
                     ,     '2021-02-11'
                     ,     'S'
                     ,     'S'
                     ,     '2021-02-11'
                     ,     1
                     ,     1
                     ,     'Sin Datos'
                     ,     'Sin Datos')
SET IDENTITY_INSERT [Sales].[DimEmployee] OFF;
GO
```

```
SET IDENTITY_INSERT [Sales].[DimProduct] ON;
INSERT INTO [Sales].[DimProduct]
                           ([ProductKey]
                  ,[ProductID]
                  ,[Name]
                  ,[ProductNumber]
                  ,[Color]
                  ,[ListPrice]
                  ,[Size]
                  ,[Weight]
                  ,[Class]
                  ,[Style]
                  ,[ProductSubcategoryID]
                  ,[ProductCategoryID]
                  ,[Subcategory]
                  ,[Category])
     VALUES
           (-1
                     ,     -1
                     ,     'Sin Datos'
                     ,     'Sin Datos'
                     ,     'Sin Datos'
                     ,     0.00
                     ,     'SinD'
                     ,     0.00
                     ,     -1
                     ,     -1
                     ,     -1
                     ,     -1
                     ,     'Sin Datos'
                     ,     'Sin Datos')

SET IDENTITY_INSERT [Sales].[DimProduct] OFF;
GO
```

```
INSERT INTO [GENERICO].[DimFecha]
           ([FechaKey]
           ,[Fecha]
           ,[Dia_Semana]
           ,[Nombre_Dia_Semana]
           ,[Dia_Mes]
           ,[Dia_Anio]
           ,[Nombre_Mes]
           ,[Mes_Anio]
           ,[Semana_Anio]
           ,[Anio]
           ,[Bimestre]
           ,[Trimestre]
           ,[Cuatrimestre]
           ,[Semestre]
           ,[Visible])
     VALUES
           (-1
           ,'1900-01-01 00:00:00.000'
           ,0
           ,'Sin Fecha'
           ,0
           ,0
           ,'Sin Fecha'
           ,0
           ,0
           ,0
           ,0
           ,0
           ,0
           ,0
           ,0
           ,1)
        GO
```

F. Implementación

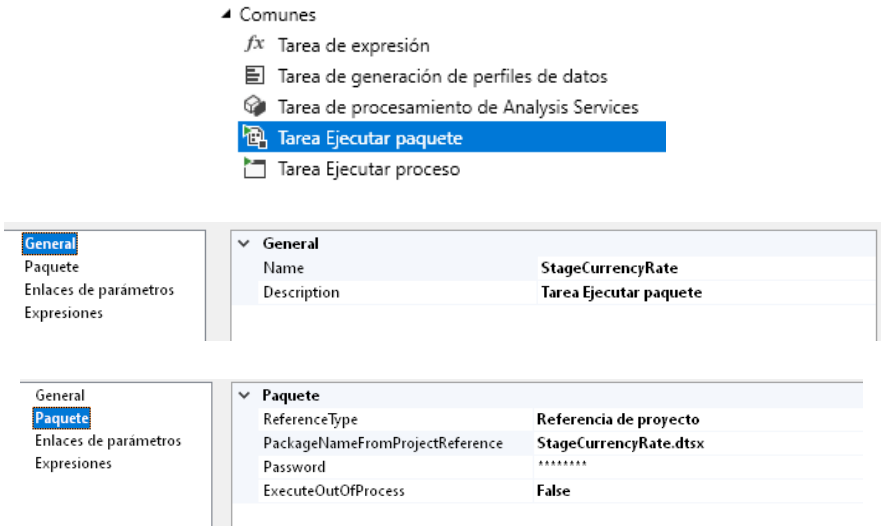
Antes de compilar y desplegar una solución para producción, es crucial crear paquetes maestros que gestionen la ejecución de las tareas en bloques. Estos paquetes permiten establecer un control de precedencias y una lógica coherente. Por ejemplo, es común que las tablas de hechos dependan de las dimensiones. Por lo tanto, antes de ejecutar las tareas relacionadas con las tablas de hechos, es esencial asegurarse de que las dimensiones estén actualizadas o pobladas adecuadamente. Esto garantiza la integridad y la coherencia de los datos en el proceso de ETL.

Para comenzar la creación de estos paquetes en los proyectos DWH_AW Y STAGE_AW debe tener en cuenta que el flujo de un paquete normal no aplica, dado que en la logica de un paquete master no se realizan conteos iniciales y/o finales, por ende, se parte desde la creación del “auditkey” y se finaliza en el “update auditkey”; estos paquetes, pueden ser copiados desde la plantilla base y posteriormente se modifican los script y asignación de parámetros derivando en el descarte del parámetro “RowCount”.

Nombre de la tarea	QUERY
Task Get AuditKey	INSERT INTO [DIM_AUDIT] ([TableName] ,[PkgName] ,[PkgGUID]

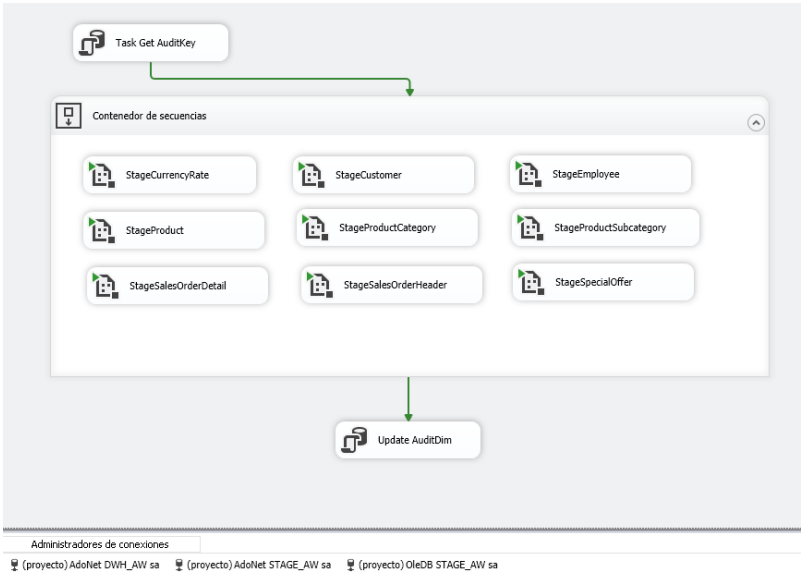
	,[ExecStartDT] ,[ExecStopDT] ,[ExecutionInstanceGUID] ,[ExtractRowCnt] ,[TableInitialRowCnt] ,[TableFinalRowCnt] ,[SuccessfulProcessingInd]) VALUES (@TableName ,@PkgName ,@PkgGUID ,@StartTime ,NULL ,@PkgExecGUID ,NULL ,NULL ,NULL ,'N') SELECT CAST(SCOPE_IDENTITY() AS INT) AS AuditKey go
Update AuditDim	UPDATE [DIM_AUDIT] SET ExecStopDT = SYSDATETIME() ,SuccessfulProcessingInd='Y' WHERE AuditKey = @AuditKey;

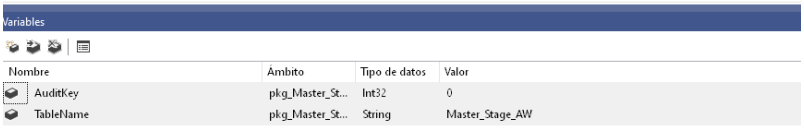
En esta sección, utilizaremos el componente "Tarea de ejecutar paquete", que consta de dos partes clave. En primer lugar, está la asignación del nombre, que generalmente coincide con el nombre del paquete. En segundo lugar, se encuentra la selección del paquete que el componente representará.

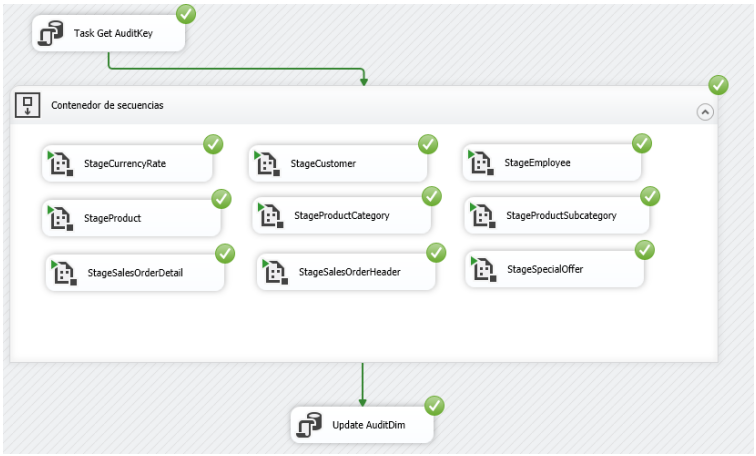


Ahora que hemos establecido cómo crear las tareas de auditoría, avanzaremos al siguiente paso: la creación de un contenedor de secuencias. Dentro de este contenedor, incluiremos las tareas de ejecución de paquetes. En este primer escenario, enfocaremos

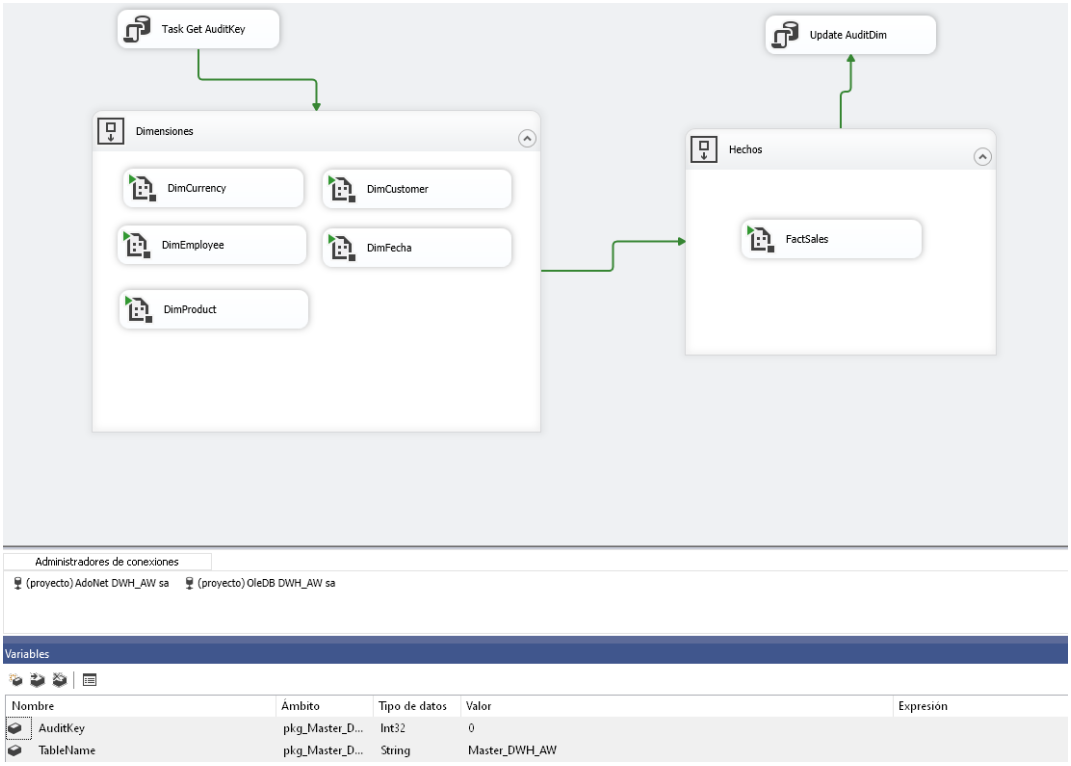
nuestro esfuerzo en la etapa "STAGE". Dada su naturaleza, podemos optar por un enfoque donde un único contenedor albergue todas las tareas relacionadas. Sin embargo, también tenemos la flexibilidad de agrupar las tareas según diversos criterios, como el proceso, la lógica, el tipo de datos, las secuencias, entre otros. La decisión sobre cómo estructurar los contenedores dependerá de la complejidad del origen de datos y las necesidades específicas del proyecto.

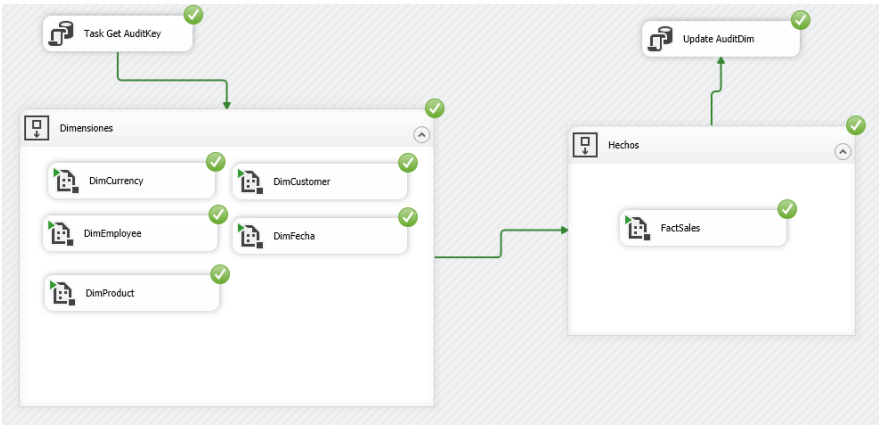






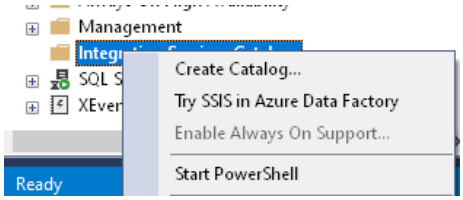
En el segundo escenario “DWH”, tenemos contenedores separados para las dimensiones y los hechos, podemos adoptar un enfoque modular y organizado. Dentro del contenedor de dimensiones, agruparemos todas las tareas relacionadas con la carga o actualización de las dimensiones. De manera similar, en el contenedor de hechos, incluiremos las tareas específicas para cargar o actualizar las tablas de hechos.

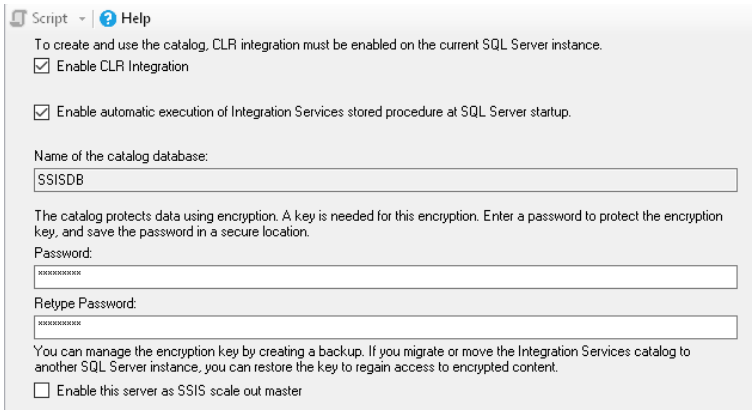




Una vez que hayamos finalizado la creación de los paquetes maestros, el siguiente paso es acceder al SQL Management Studio. Desde allí, nos dirigiremos a la sección de Integration Services para establecer el catálogo "SSISDB". Este catálogo nos proporciona la infraestructura necesaria para implementar y gestionar los paquetes.

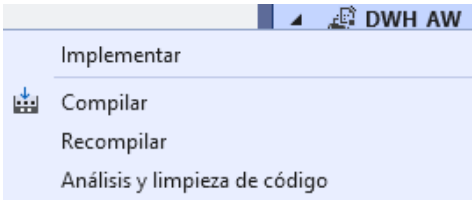
La creación del catálogo "SSISDB" implica dos pasos clave: habilitar la Integración CLR y asignar una clave para la encriptación de los paquetes. Estos pasos son fundamentales para garantizar la seguridad y el correcto funcionamiento de los paquetes en el entorno de producción. A continuación, detallaremos cómo llevar a cabo estos procesos dentro del SQL Management Studio:

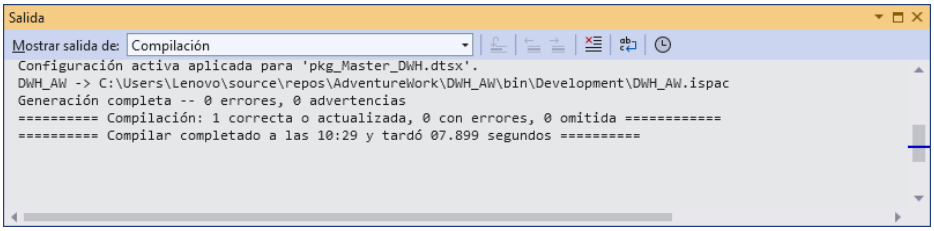




Después de configurar el catálogo SSISDB en SQL Server, el siguiente paso es compilar cada proyecto, "DWH_AW" y "STAGE_AW". Esta acción nos permite detectar y corregir posibles errores antes de la implementación en el entorno de producción.

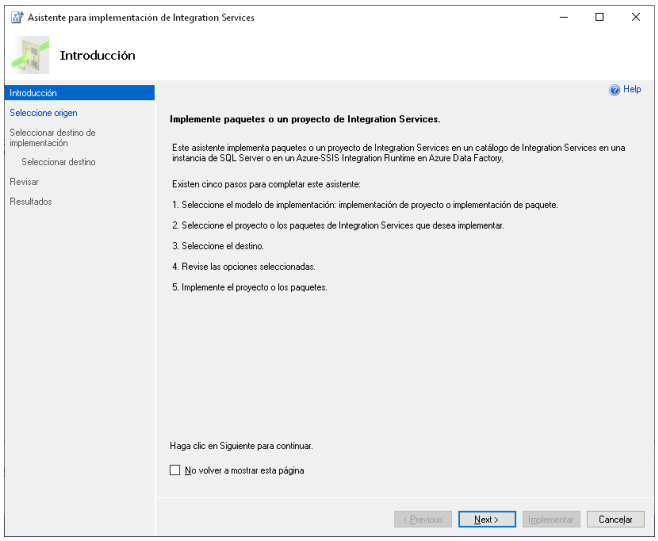
- Build (Compilar): Este proceso genera el código ejecutable a partir de los paquetes SSIS y otros objetos del proyecto. Durante la compilación, se verifica la sintaxis de los paquetes, se resuelven las referencias a componentes y se generan los archivos binarios necesarios para ejecutar los paquetes.
- Rebuild (Recompilar): Similar a la compilación, el proceso de reconstrucción elimina los archivos compilados existentes y genera nuevos archivos binarios desde cero. A diferencia de la compilación normal, que solo actualiza los archivos que han cambiado, la reconstrucción regenera todo el código ejecutable sin tener en cuenta el estado anterior.

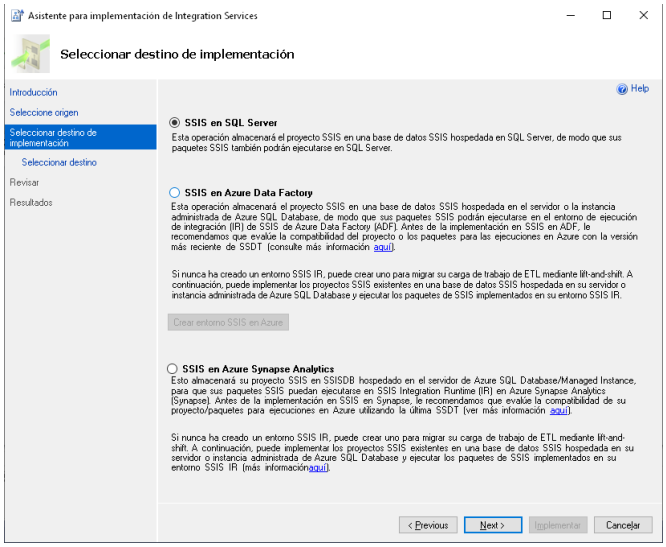




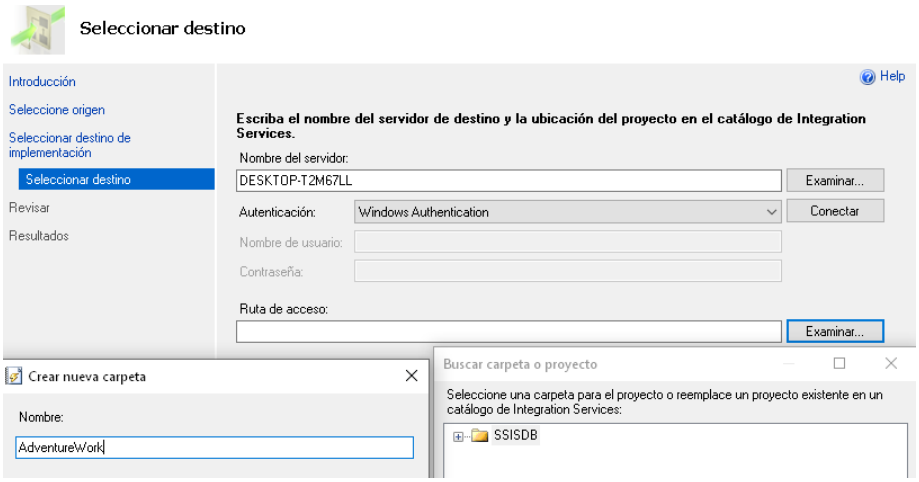
Una vez que los proyectos están en perfecto estado, seleccionamos la opción "Implementar", lo que desencadena una serie de ventanas emergentes. Estas incluyen una

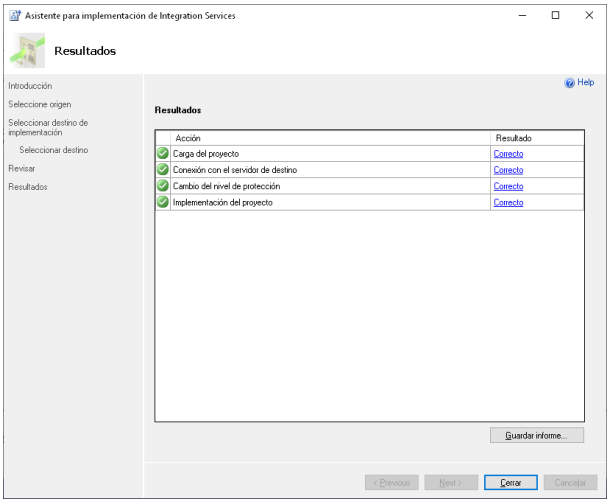
introducción general, la especificación del destino de implementación (SSIS en SQL Server) y la selección del destino específico.



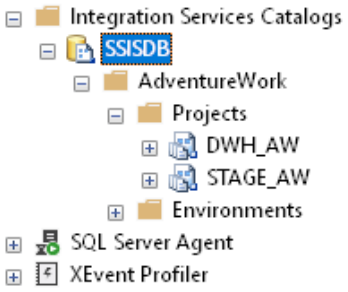


En la sección "Seleccionar destino", ingresaremos la dirección IP o el nombre del servidor (en este ejercicio, será "local"). Utilizaremos la autenticación de tipo Windows y procederemos a realizar la conexión. Una vez completados estos pasos, se habilitará la opción "Examinar". Aquí, crearemos una nueva carpeta con el nombre de la solución y seleccionaremos únicamente esa carpeta. Finalmente seguimos con la implementacion y cerramos el cuadro, repetimos este ejercicio para ambos proyectos.

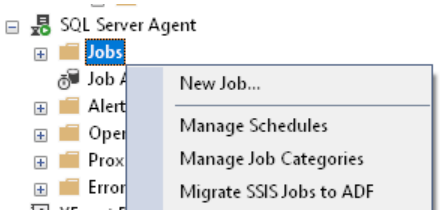




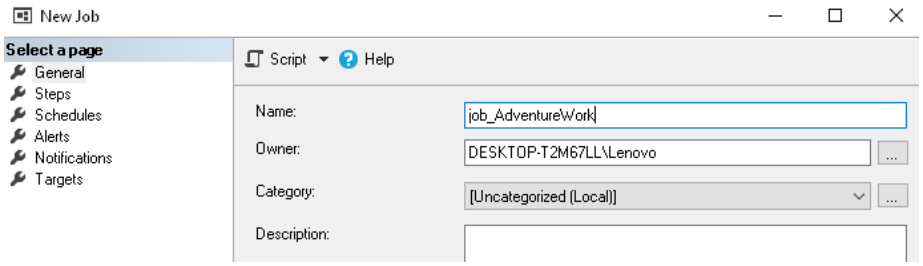
Al dirigirnos al SQL Server Management Studio, podrás observar la carpeta creada desde SSIS y los proyectos implementados en ella. Esto te permitirá acceder y gestionar los paquetes implementados directamente desde el entorno de gestión de SQL Server.



Finalmente, procedemos con la creación de un trabajo (job) desde el apartado de SQL Server Agent. Esto automatiza la tarea relacionada con AdventureWorks, lo que garantiza su ejecución de forma programada y sin intervención manual.

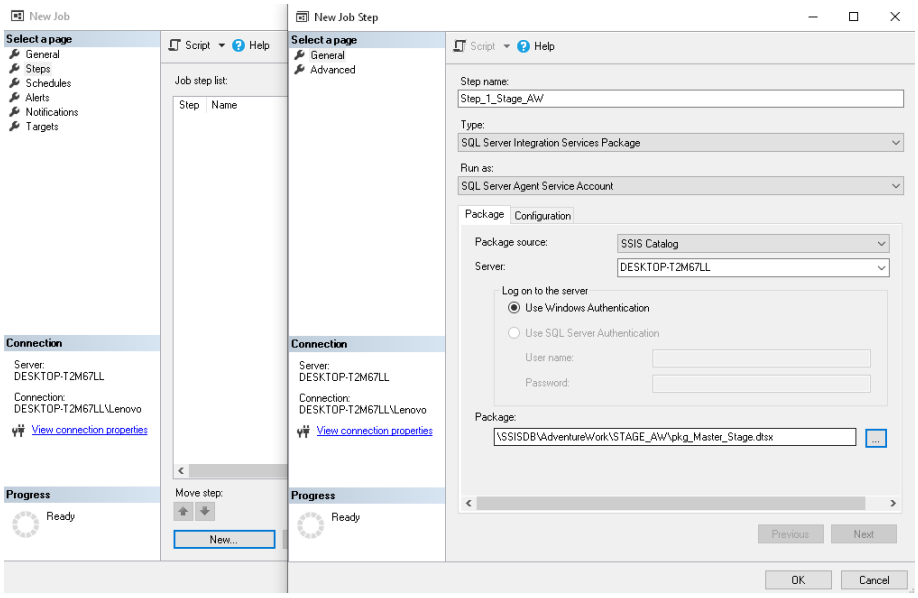


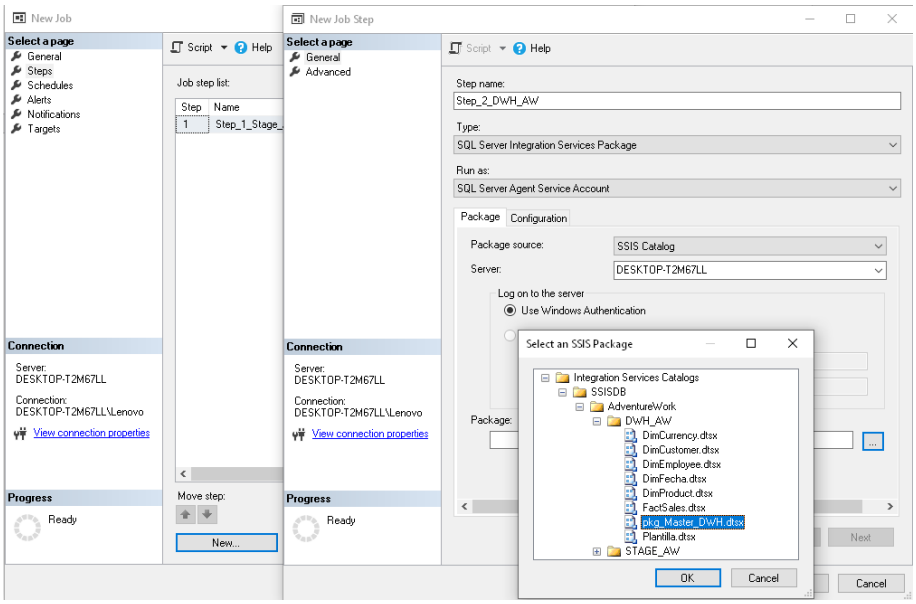
Al crear el trabajo (job), se nos solicita proporcionar un nombre para identificarlo y especificar el propietario (owner).



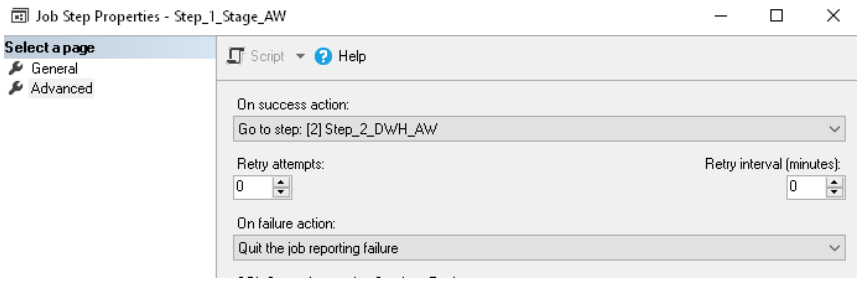
En la opción "Pasos" (Steps), procedemos a crear los pasos ordenados que se ejecutarán para cumplir con la lógica previamente establecida en SSIS. Esto implica asignar un nombre al paso, seleccionar el tipo "Paquete de Integration Services de SQL Server" (SQL Server Integration Services Package), elegir el servidor donde se compiló la solución y, en el apartado de "Paquete" (Package), buscar el paquete maestro del proyecto.

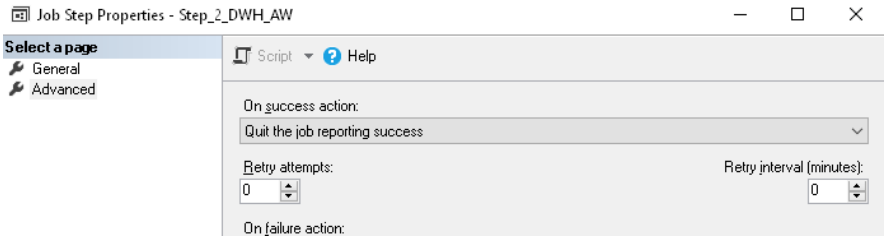
- Step_1_Stage_AW
- Step_2_DWH_AW

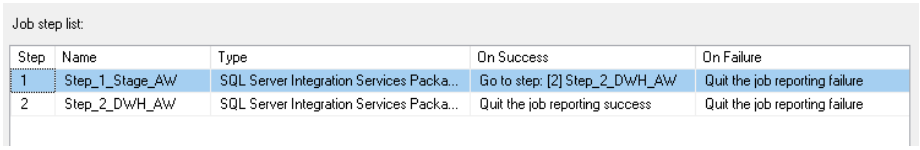




Al concluir con la creación de los pasos, es necesario editarlos para establecer dependencias de ejecución. Por defecto, estas dependencias siguen el orden de creación o el orden de su posición en la lista de pasos. Para asignar una dependencia, accedemos a cada paso y modificamos el apartado de "Avanzado", donde especificamos la secuencia a seguir, es decir, el paso que debe ejecutarse despues. En el caso del último paso, podemos establecer la opción "Salir del trabajo informando éxito" ("Quit the job reporting success"). Esto garantiza que el trabajo finalice correctamente cuando se complete el último paso.







Finalizamos creando el cronograma de ejecución que seguirá el trabajo en la sección "Horarios" (Schedules). Aquí, asignaremos un nombre a la rutina y procederemos a configurar la frecuencia de ejecución. Esto puede incluir horarios específicos, días de la

semana, intervalos de repetición, entre otros parámetros según sea necesario para cumplir con los requerimientos del proyecto.

